Український католицький університет

Statistics and Econometrics

Завдання 1.

Описова статистика та елементи теорії ймовірностей

Владислав Григола

Аналітичний звіт

Задача 1. Реальні дані (ceosal1)

Дослідіть дані про оплату праці виконавчих директорів (CEO) США та показники їхніх компаній:

- `salary` — річна заробітна плата разом із бонусами, тис. доларів США;
- `pcsalary` — відсоткова зміна заробітної плати порівняно з попереднім роком;
- `age` — вік CEO у роках, змодельований за розподілом $N(53, 6.5^2)$;
- `sales` — річний дохід компанії від продажів, млн доларів США;
- `roe` — рентабельність власного капіталу компанії, %.

Вік є синтетичним: його немає у вихідному наборі `ceosal1`. Висновки про зв'язок віку з іншими показниками стосуються змодельованих даних. Початкове значення генератора `SEED = 42` забезпечує відтворюваність. Дані завантажуються під час першого запуску та зберігаються локально в `data/`.

Для відтворюваності обчислень використано Python та бібліотеки NumPy, pandas, SciPy, matplotlib і seaborn. Початкове значення генератора випадкових чисел: `SEED = 42`.

```
from statsmodels.datasets import get_rdataset

cache = DATA_DIR / "ceosal1.csv"
if cache.exists():
    df = pd.read_csv(cache)
else:
    df = get_rdataset("ceosal1", "wooldridge").data
    df.to_csv(cache, index=False)

# Синтетичний вік, а не спостережувані дані. Скидаємо генератор для відтворюваності повторних запусків.
df["age"] = np.random.default_rng(SEED).normal(53, 6.5, size=len(df))
columns = ["salary", "pcsalary", "age", "sales", "roe"]
assert set(columns).issubset(df.columns)
df[columns].head()
```

	salary	pcsalary	age	sales	roe
0	1095	20	54.980661	27595.000000	14.1
1	1001	32	46.240103	9958.000000	10.9
2	1122	9	57.877933	6125.899902	23.5
3	578	-9	59.113671	16246.000000	5.9
4	1368	7	40.318271	21783.199219	13.8

Частина 1(a). Міри положення

Для `salary` обчисліть середнє значення, 5%-усічене середнє, медіану, нижній і верхній квартилі та квантилі рівнів 0.05 і 0.95. Дайте економічну інтерпретацію кожного показника — щонайменше одне речення на показник. Зазначте, яку частку спостережень відкидаєте з кожного краю під час обчислення усіченого середнього.

```
salary = df["salary"].dropna()

# всього спостережень
n = len(salary)

proportiontocut = 0.05

# кількість спостережень, які треба відкинути з кожного краю вибірки
k = int(np.floor(proportiontocut * n))

mean_salary = salary.mean()
trimmed_mean_5 = stats.trim_mean(salary, proportiontocut=proportiontocut)
median_salary = salary.median()

q05 = salary.quantile(0.05)
q25 = salary.quantile(0.25)
q75 = salary.quantile(0.75)
q95 = salary.quantile(0.95)
```

```
results = pd.Series(
    {
        "Середнє": mean_salary,
        "5%-усічене середнє": trimmed_mean_5,
        "Нижній 5%-квантиль": q05,
        "Нижній квартиль (Q1)": q25,
        "Медіана": median_salary,
        "Верхній квартиль (Q3)": q75,
        "Верхній 5%-квантиль": q95,
    },
    name="Значення, тис. дол. США",
)

print(f"Кількість спостережень: n = {n}")
print(f"Для 5%-усіченого середнього відкидаємо {k} спостережень з кожного краю.")

results.to_frame().round(2)
```

```
Кількість спостережень: n = 209
Для 5%-усіченого середнього відкидаємо 10 спостережень з кожного краю.
```

	Значення, тис. дол. США
Середнє	1281.12
5%-усічене середнє	1100.88
Нижній 5%-квантиль	450.00
Нижній квартиль (Q1)	736.00
Медіана	1039.00
Верхній квартиль (Q3)	1407.00
Верхній 5%-квантиль	2295.40

Інтерпретація

- **Середнє:** середня річна зарплата CEO у вибірці становить **1281.12 тис. дол. США**.
- **5%-усічене середнє:** після вилучення 10 найменших і 10 найбільших значень зарплати середнє становить **1100.88 тис. дол. США**. Воно нижче за звичайне середнє, отже дуже високі зарплати помітно збільшують середнє значення.
- **Медіана:** медіанна зарплата становить **1039 тис. дол. США**. Приблизно половина CEO має зарплату нижчу або рівну цьому значенню, а інша половина – вищу або рівну.
- **Нижній квартиль Q_1:** приблизно 25% CEO мають зарплату не більше **736 тис. дол. США**.
- **Верхній квартиль Q_3:** приблизно 75% CEO мають зарплату не більше **1407 тис. дол. США**, а близько 25% мають більшу зарплату.
- **5%-квантиль:** приблизно 5% CEO мають зарплату не більше **450 тис. дол. США**.
- **95%-квантиль:** приблизно 95% CEO мають зарплату не більше **2295.40 тис. дол. США**, а близько 5% мають більшу зарплату.

Звичайне середнє (**1281.12**) помітно вище за медіану (**1039**) і 5%-усічене середнє (**1100.88**). Це показує, що в даних є відносно невелика кількість дуже високих зарплат, які підтягують середнє вгору.

Частина 1(б). Емпірична функція розподілу

Побудуйте емпіричну кумулятивну функцію розподілу (ECDF) для `salary`. Обчисліть та поясніть в економічних термінах:

1. $\widehat{F}^{-1}(0.1)$ та $\widehat{F}^{-1}(0.9)$;
2. $\widehat{F}(1000)$ та $1 - \widehat{F}(2500)$.

Зазначте використане означення вибіркового квантиля. Пам'ятайте, що `salary` вимірюється в тисячах доларів США.

```
# Перетворюємо Series у масив NumPy
salary_values = salary.to_numpy()

# Сортуємо зарплати від найменшої до найбільшої
salary_sorted = np.sort(salary_values)

# Для кожного відсортованого значення визначаємо накопичену частку:
# 1/n, 2/n, ..., n/n
ecdf_y = np.arange(1, n + 1) / n


# --- Квантилі ---

# Використовуємо означення квантиля з лекції:
# якщо np не ціле — беремо наступне впорядковане значення;
# якщо np ціле — усереднюємо два сусідні значення.
q10, q90 = np.quantile(
    salary_values,
    [0.10, 0.90],
    method="averaged_inverted_cdf"
)


# --- Значення ECDF ---

# Частка CEO із зарплатою <= 1000 тис. дол. США
F_1000 = np.mean(salary_values <= 1000)

# Частка CEO із зарплатою <= 2500 тис. дол. США
F_2500 = np.mean(salary_values <= 2500)

# Частка CEO із зарплатою > 2500 тис. дол. США
above_2500 = 1 - F_2500


# --- Виведення результатів ---

print(f"F^(-1)(0.1) = {q10:.2f} тис. дол. США")
print(f"F^(-1)(0.9) = {q90:.2f} тис. дол. США")
print(f"F(1000) = {F_1000:.4f} ({F_1000 * 100:.2f}%)")
print(f"1 - F(2500) = {above_2500:.4f} ({above_2500 * 100:.2f}%)")


# --- Побудова ECDF ---

plt.figure(figsize=(9, 5))

plt.step(
    salary_sorted,
    ecdf_y,
    where="post"
)

plt.title("Емпірична функція розподілу зарплат CEO")
plt.xlabel("Річна зарплата CEO, тис. дол. США")
plt.ylabel("Накопичена частка спостережень")

plt.ylim(0, 1.02)
plt.grid(True)

plt.tight_layout()
plt.show()
```

```
F^(-1)(0.1) = 525.00 тис. дол. США
F^(-1)(0.9) = 1900.00 тис. дол. США
F(1000) = 0.4785 (47.85%)
1 - F(2500) = 0.0431 (4.31%)
```



Інтерпретація

- $\hat{F}^{-1}(0.1) = 525$ **тис. дол. США:** приблизно 10% CEO у вибірці мають річну зарплату не більше **525 тис. дол. США**.
- $\hat{F}^{-1}(0.9) = 1900$ **тис. дол. США:** приблизно 90% CEO мають річну зарплату не більше **1900 тис. дол. США**, а приблизно 10% — більше.
- $\hat{F}(1000) = 0.4785$**:** приблизно **47.85%** CEO у вибірці мають річну зарплату не більше **1000 тис. дол. США (1 млн дол. США)**.
- $1 - \hat{F}(2500) = 0.0431$**:** приблизно **4.31%** CEO мають річну зарплату більше **2500 тис. дол. США (2.5 млн дол. США)**.

Графік ECDF швидко зростає в області нижчих зарплат і далі зростає значно повільніше. Це означає, що більшість зарплат зосереджена у нижчому діапазоні, а дуже високі зарплати зустрічаються рідко.

Частина 1(в). Гістограма та діаграма розмаху

Побудуйте гістограму та діаграму розмаху для `salary` зі стандартними налаштуваннями `matplotlib` або `seaborn`. Поясніть, як інтерпретується площа прямокутника гістограми з обраним типом нормування. Пов'яжіть елементи діаграми розмаху з обчисленими вище мірами положення.

```
# --- Гістограма ---

plt.figure(figsize=(9, 5))

sns.histplot(salary)

plt.title("Розподіл річних зарплат CEO")
plt.xlabel("Річна зарплата CEO, тис. дол. США")
plt.ylabel("Кількість спостережень")
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

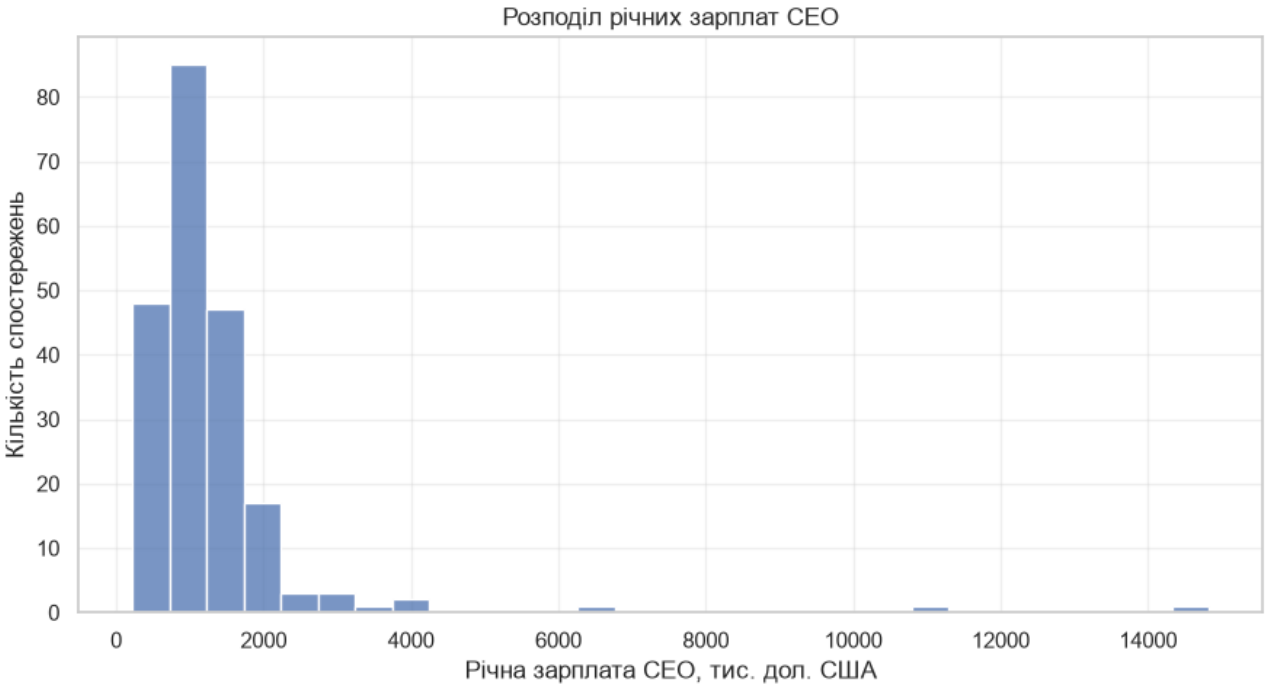

# --- Діаграма розмаху ---
```

```
plt.figure(figsize=(9, 3))

sns.boxplot(
    x=salary
)

plt.title("Діаграма розмаху річних зарплат CEO")
plt.xlabel("Річна зарплата CEO, тис. дол. США")
plt.grid(True, axis="x", alpha=0.3)

plt.tight_layout()
plt.show()
```





Інтерпретація

Гістограма показує, що більшість зарплат CEO зосереджена приблизно в діапазоні до 2000 тис. дол. США. Водночас є невелика кількість дуже високих зарплат, через що розподіл має довгий правий хвіст.

За стандартних налаштувань `sns.histplot()` висота стовпчика показує кількість спостережень у відповідному інтервалі. Площа прямокутника дорівнює цій кількості, помноженій на ширину інтервалу. Оскільки тут інтервали однакової ширини, площі пропорційні кількостям спостережень. Щоб отримати частку спостережень, потрібно поділити площу на ширину інтервалу та загальну кількість спостережень.

Діаграма розмаху узгоджується з раніше обчисленими показниками: - нижній квартиль $Q_1 = 736$ тис. дол. США; - медіана = 1039 тис. дол. США; - верхній квартиль $Q_3 = 1407$ тис. дол. США.

Отже, центральні 50% зарплат знаходяться приблизно між **736** та **1407 тис. дол. США**.

На boxplot також видно кілька значень значно вище основної частини вибірки. Вони позначені як потенційні викиди. Саме такі дуже високі зарплати пояснюють довгий правий хвіст гістограми і те, що середнє значення зарплати вище за медіану.

Частина 1(г). Форма розподілу

Зробіть висновок про розподіл `salary`. Чи залишаються доречними обчислені вище міри положення? Обчисліть коефіцієнт асиметрії Фішера–Пірсона та поясніть його значення.

```
# Емпіричне стандартне відхилення.
# ddof=0 означає, що у формулі ділимо на n.
salary_std_empirical = salary.std(ddof=0)

# Коефіцієнт асиметрії Фішера–Пірсона:
# для кожного спостереження знаходимо відхилення від середнього,
# ділимо на стандартне відхилення, підносимо до куба
# і беремо середнє цих значень.
salary_skewness = np.mean(
    ((salary - mean_salary) / salary_std_empirical) ** 3
)

print(f"Коефіцієнт асиметрії Фішера–Пірсона = {salary_skewness:.3f}")

# Перевірка через scipy: bias=True відповідає формулі
# через емпіричні центральні моменти з дільником n.
salary_skewness_scipy = stats.skew(salary, bias=True)

print(f"Перевірка через scipy = {salary_skewness_scipy:.3f}")
```

```
Коефіцієнт асиметрії Фішера–Пірсона = 6.855
Перевірка через scipy = 6.855
```

Інтерпретація

Коефіцієнт асиметрії Фішера–Пірсона дорівнює **6.855**. Він є додатним і досить великим, тому розподіл зарплат має **сильну правосторонню асиметрію**.

Це узгоджується з гістограмою та boxplot: більшість зарплат знаходяться у значно нижчому діапазоні, але є кілька дуже високих значень, які утворюють довгий правий хвіст.

Через ці високі зарплати середнє значення (**1281.12 тис. дол. США**) помітно вище за медіану (**1039 тис. дол. США**). Тому для опису типового рівня зарплати медіана та 5%-усічене середнє є більш надійними, ніж звичайне середнє.

Частина 1(ґ). Вибір інтервалів гістограми

Перевірте й коротко опишіть правило, за яким використана бібліотека у встановленій версії визначає стандартну ширину інтервалу або кількість стовпчиків гістограми. Побудуйте гістограми з `bins=150` та `bins=3`. Поясніть, що показує або приховує надмірна деталізація та надмірне узагальнення.

```
# Межі інтервалів, які NumPy вибирає автоматично
auto_bin_edges = np.histogram_bin_edges(salary, bins="auto")

# Кількість інтервалів на 1 менша за кількість їхніх меж
auto_bin_count = len(auto_bin_edges) - 1

print(f"Автоматично вибрана кількість інтервалів: {auto_bin_count}")
```

```
Автоматично вибрана кількість інтервалів: 29
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Занадто детальна гістограма ---

sns.histplot(
    salary,
    bins=150,
    stat="density",
    edgecolor="black",
    ax=axes[0]
)

axes[0].set_title("Занадто детальна гістограма (bins=150)")
axes[0].set_xlabel("Річна зарплата CEO, тис. дол. США")
axes[0].set_ylabel("Щільність")
axes[0].grid(True, alpha=0.3)

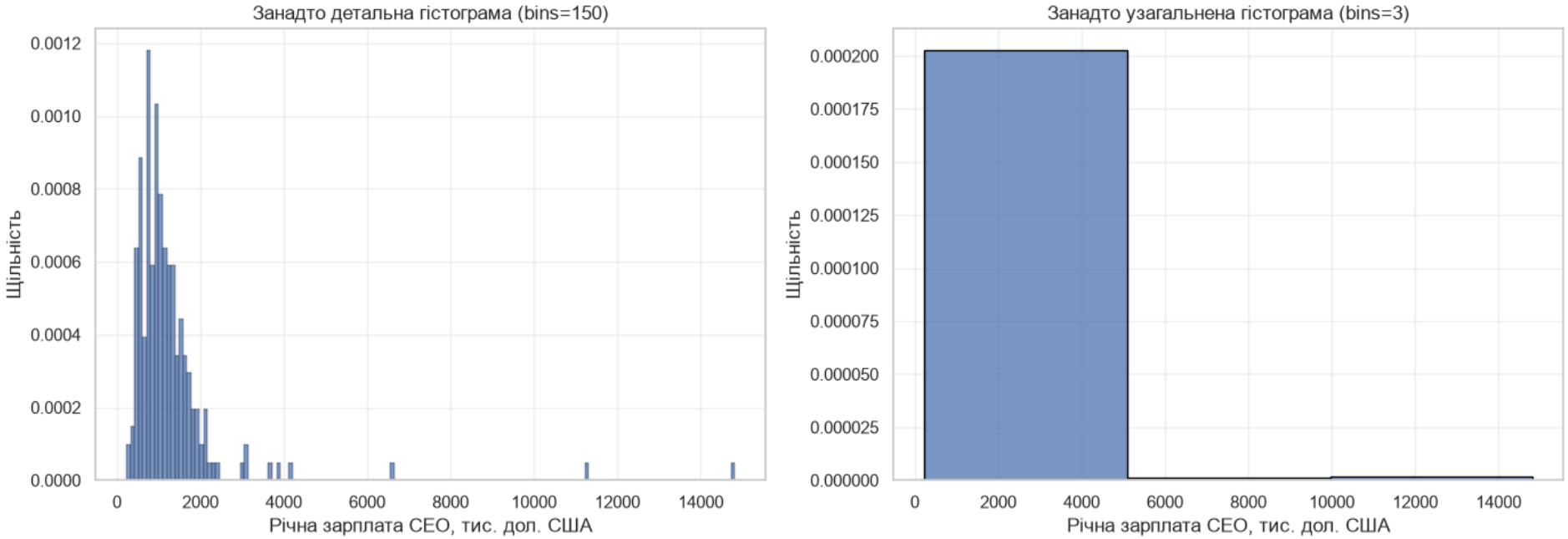

# --- Занадто узагальнена гістограма ---

sns.histplot(
    salary,
    bins=3,
    stat="density",
    edgecolor="black",
    ax=axes[1]
)

axes[1].set_title("Занадто узагальнена гістограма (bins=3)")
axes[1].set_xlabel("Річна зарплата CEO, тис. дол. США")
axes[1].set_ylabel("Щільність")
axes[1].grid(True, alpha=0.3)


plt.tight_layout()
plt.show()
```

Порівняння деталізації гістограм



Зарплата вимірюється в тис. дол. США; вертикальні осі показують щільність.

Інтерпретація

За стандартних налаштувань `sns.histplot()` використовує `bins="auto"`. У нашому випадку автоматично було вибрано **29 інтервалів**.

Режим `auto` у NumPy порівнює правило Стерджеса та правило Фрідмана–Діаконіса і використовує більш деталізований варіант. Мета такого автоматичного вибору — отримати достатньо детальну гістограму, але не перевантажити її випадковими коливаннями.

При `bins=150` гістограма стає занадто детальною: з'являється багато дуже вузьких стовпчиків, частина з яких містить лише одне або кілька спостережень. Через це графік виглядає шумним і загальну форму розподілу бачити складніше.

При `bins=3` гістограма, навпаки, занадто узагальнена. Майже всі звичайні зарплати потрапляють в один широкий інтервал, тому ми втрачаємо інформацію про те, як саме зарплати розподілені в основній частині вибірки.

Отже, кількість інтервалів суттєво впливає на вигляд гістограми: занадто велика кількість показує надмірний шум, а занадто мала приховує важливу структуру даних. Автоматично вибрані 29 інтервалів дають значно корисніший компроміс для цієї вибірки.

Частина 1(д). Логарифмування заробітної плати

Розгляньте натуральний логарифм $\ln(\text{salary})$. Побудуйте гістограму та діаграму розмаху й порівняйте їх із графіками вихідної змінної. Обчисліть середнє та медіану логарифмованих значень. Зробіть висновки про симетричність розподілу та надайте економічну інтерпретацію результатів.

```
# Створюємо логарифмовані значення зарплати.
# Початкову змінну salary не змінюємо.
log_salary = np.log(salary)

# Середнє та медіана логарифмованих зарплат
log_mean_salary = log_salary.mean()
log_median_salary = log_salary.median()

# Коефіцієнт асиметрії після логарифмування.
# Використовуємо той самий підхід, що й для початкових зарплат.
log_salary_std_empirical = log_salary.std(ddof=0)

log_salary_skewness = np.mean(
    ((log_salary - log_mean_salary) / log_salary_std_empirical) ** 3
)

print(f"Середнє ln(salary) = {log_mean_salary:.3f}")
print(f"Медіана ln(salary) = {log_median_salary:.3f}")
print(f"Коефіцієнт асиметрії ln(salary) = {log_salary_skewness:.3f}")
# Повертаємо середнє логарифмів до грошової шкали.
geometric_mean_salary = np.exp(log_mean_salary)
print(f"Геометричне середнє зарплати = {geometric_mean_salary:.2f} тис. дол. США")
```

```
Середнє ln(salary) = 6.950
Медіана ln(salary) = 6.946
Коефіцієнт асиметрії ln(salary) = 0.880
Геометричне середнє зарплати = 1043.55 тис. дол. США
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))


# --- Гістограма логарифмованої зарплати ---

sns.histplot(
    log_salary,
    bins="auto",
    stat="density",
    edgecolor="black",
    ax=axes[0]
)
```

```
axes[0].set_title("Розподіл ln(salary)")
axes[0].set_xlabel("ln(річної зарплати CEO)")
axes[0].set_ylabel("Щільність")
axes[0].grid(True, alpha=0.3)

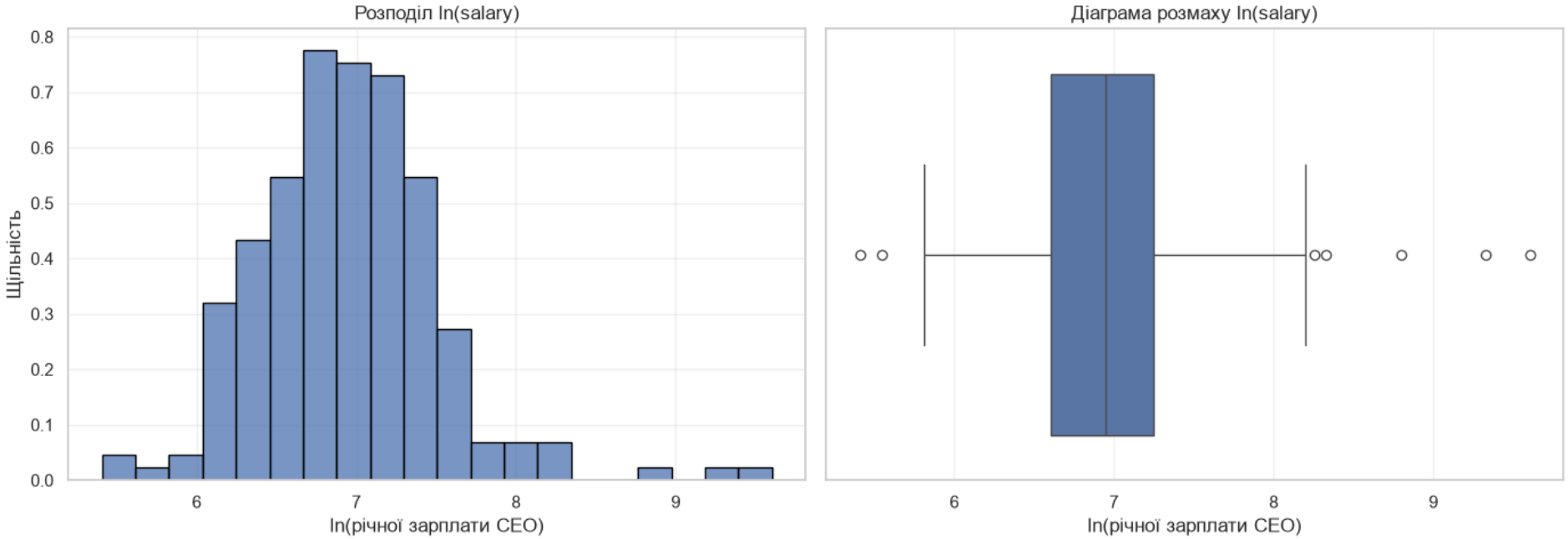

# --- Діаграма розмаху логарифмованої зарплати ---

sns.boxplot(
    x=log_salary,
    ax=axes[1]
)

axes[1].set_title("Діаграма розмаху ln(salary)")
axes[1].set_xlabel("ln(річної зарплати CEO)")
axes[1].grid(True, axis="x", alpha=0.3)


plt.tight_layout()
plt.show()
```

Розподіл логарифмованої зарплати



На горизонтальних осях — натуральний логарифм зарплати в тис. дол. США; гістограма показує щільність.

Інтерпретація

Після логарифмування розподіл зарплат став значно більш симетричним. Це видно і на гістограмі, і за коефіцієнтом асиметрії: він зменшився з **6.855** до **0.880**.

Коефіцієнт асиметрії все ще додатний, тому невелика правостороння асиметрія залишається, але вона вже набагато слабша.

Середнє значення `ln(salary)` дорівнює **6.950**, а медіана — **6.946**. Вони майже однакові, що також показує, що після логарифмування розподіл став ближчим до симетричного.

Якщо піднести e до середнього логарифма зарплати, отримаємо геометричне середнє — **1043.55 тис. дол. США**. Воно описує середній рівень зарплати на логарифмічній шкалі та менше залежить від дуже великих зарплат, ніж звичайне середнє.

Якщо повернути медіану назад до початкової шкали:

$$e^{6.946} \approx 1039$$

тис. дол. США, що відповідає медіані початкових зарплат.

Логарифмування стискає дуже великі значення зарплат, тому окремі дуже високооплачувані CEO вже не так сильно розтягують правий хвіст розподілу.

Частина 2(a). Кореляція Пірсона

Ми припускаємо, що заробітна плата CEO та інші змінні компанії пов’язані між собою. Обчисліть коефіцієнти кореляції Пірсона та зобразіть їх у вигляді теплової карти (кореляційної карти). Обговоріть силу та напрямок кореляцій.

```
# Змінні, між якими аналізуємо кореляцію.
# Використовуємо початкові значення без логарифмування.
analysis_columns = ["salary", "pcsalary", "age", "sales", "roe"]

# Обчислюємо матрицю коефіцієнтів кореляції Пірсона.
# Кожна комірка показує кореляцію між відповідною парою змінних.
pearson_corr = df[analysis_columns].corr(method="pearson")

# Виводимо числові значення для точного перегляду.
display(pearson_corr.round(3))

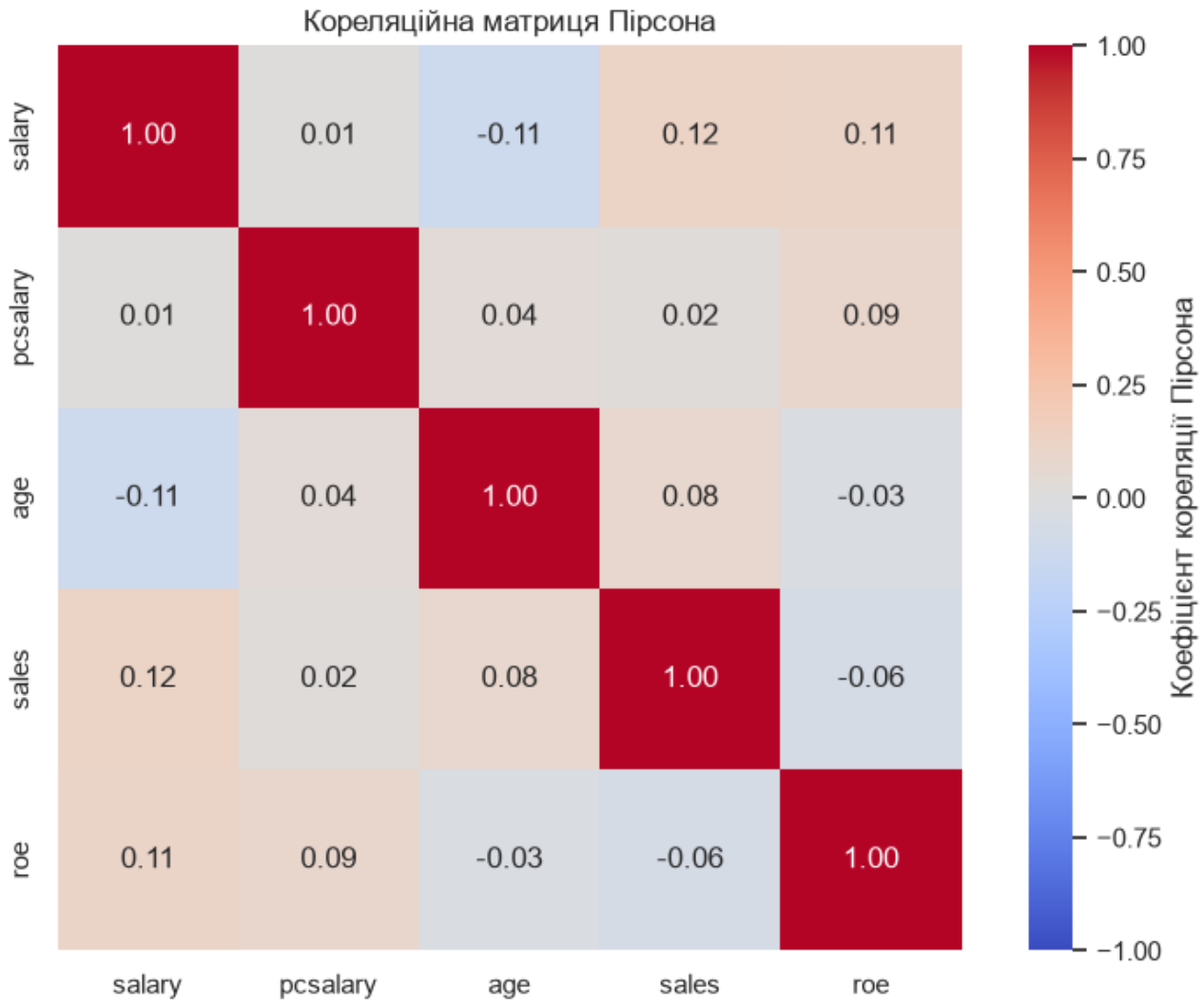

# --- Теплова карта кореляцій ---

plt.figure(figsize=(8, 6))

sns.heatmap(
    pearson_corr,
    annot=True,          # показуємо числове значення у кожній комірці
    fmt=".2f",           # округлення до двох знаків
    vmin=-1,
    vmax=1,
    center=0,
    cmap="coolwarm",
    square=True,
    cbar_kws={"label": "Коефіцієнт кореляції Пірсона"}
)

plt.title("Кореляційна матриця Пірсона")
plt.tight_layout()
plt.show()
```

	salary	pcsalary	age	sales	roe
salary	1.000	0.009	-0.110	0.120	0.115
pcsalary	0.009	1.000	0.036	0.017	0.087
age	-0.110	0.036	1.000	0.080	-0.027
sales	0.120	0.017	0.080	1.000	-0.055
roe	0.115	0.087	-0.027	-0.055	1.000



Інтерпретація

Усі отримані коефіцієнти кореляції Пірсона за модулем є дуже малими: найбільше значення становить лише близько **0.12**. Отже, у цих даних немає помітних сильних лінійних зв'язків між розглянутими змінними.

Для `salary`:

- `salary` і `sales`: $r \approx 0.12$ — дуже слабкий позитивний лінійний зв'язок;
- `salary` і `roe`: $r \approx 0.11$ — дуже слабкий позитивний зв'язок;
- `salary` і `age`: $r \approx -0.11$ — дуже слабкий негативний зв'язок;
- `salary` і `pcsalary`: $r \approx 0.01$ — практично відсутній лінійний зв'язок.

Інші пари змінних також мають коефіцієнти, близькі до нуля, тому сильних лінійних залежностей між ними не видно.

Значення **1.00** на головній діагоналі означають кореляцію кожної змінної із самою собою і окремо не інтерпретуються.

Отже, за кореляцією Пірсона жодна з розглянутих змінних не має сильного лінійного зв’язку із зарплатою CEO.

Частина 2(б). Діаграми розсіювання

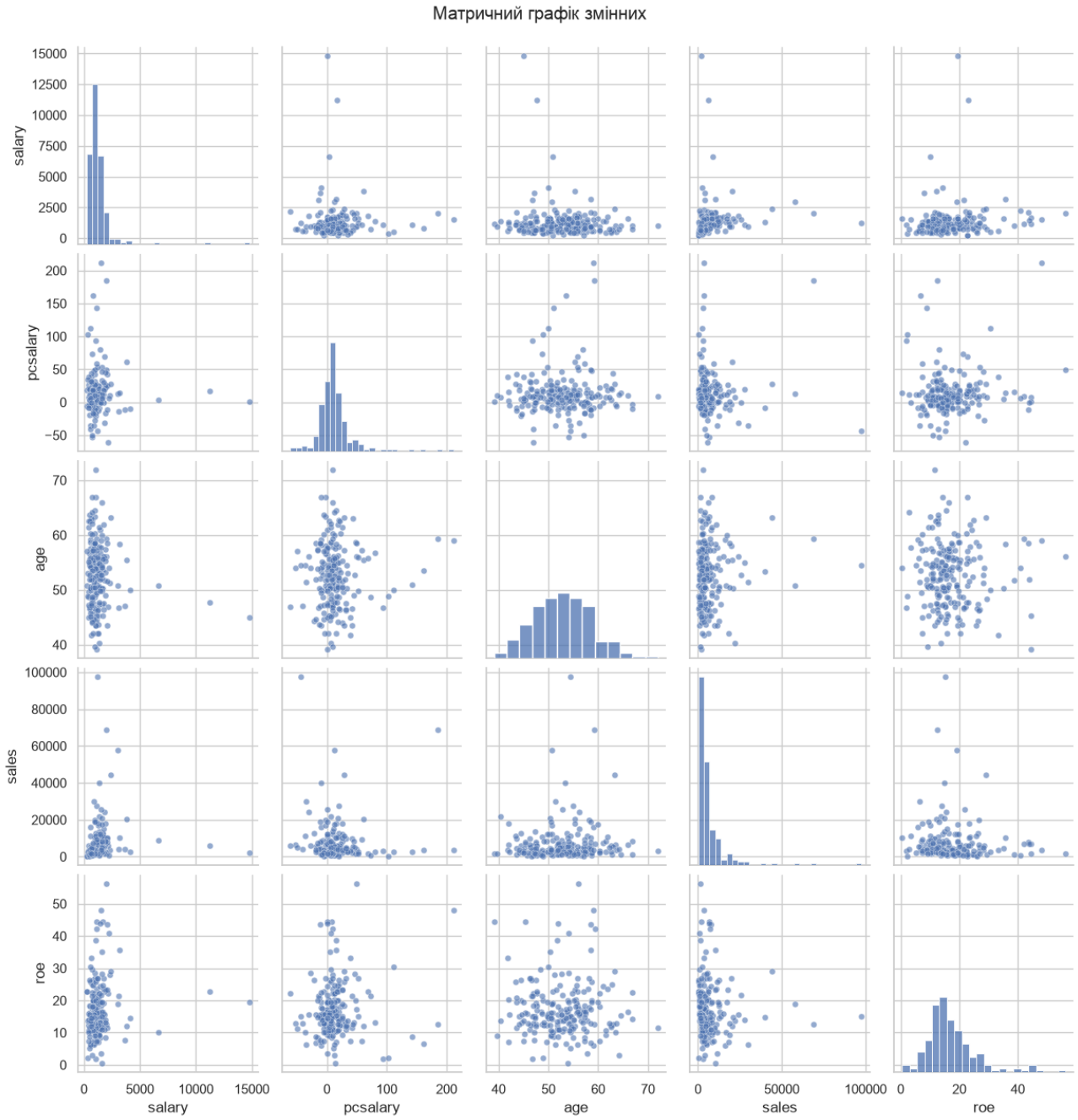
Побудуйте діаграми розсіювання (матричний графік `pairplot` у Python). Зробіть висновок, чи доречні тут лінійні коефіцієнти кореляції. Тепер обчисліть кореляції Спірмена та побудуйте для них теплову карту. Порівняйте отримані результати з результатами Пірсона.

```
# Будуємо матрицю діаграм розсіювання для всіх пар змінних.
# На діагоналі показуються розподіли окремих змінних.
sns.pairplot(
    df[analysis_columns],
    diag_kind="hist",
    plot_kws={
        "alpha": 0.6,  # робимо точки трохи прозорими
        "s": 25        # розмір точок
    }
)

plt.suptitle(
    "Матричний графік змінних",
    y=1.02
)

plt.show()
```

Матриця діаграм розсіювання



Одиниці вимірювання: salary — тис. дол. США; sales — млн дол. США; age — роки; pcsalary та roe — %. Вік age змодельовано синтетично.

```
# Обчислюємо матрицю кореляцій Спірмена.
# Метод працює з рангами значень, а не безпосередньо з їх величинами.
spearman_corr = df[analysis_columns].corr(method="spearman")

# Виводимо точні числові значення
display(spearman_corr.round(3))

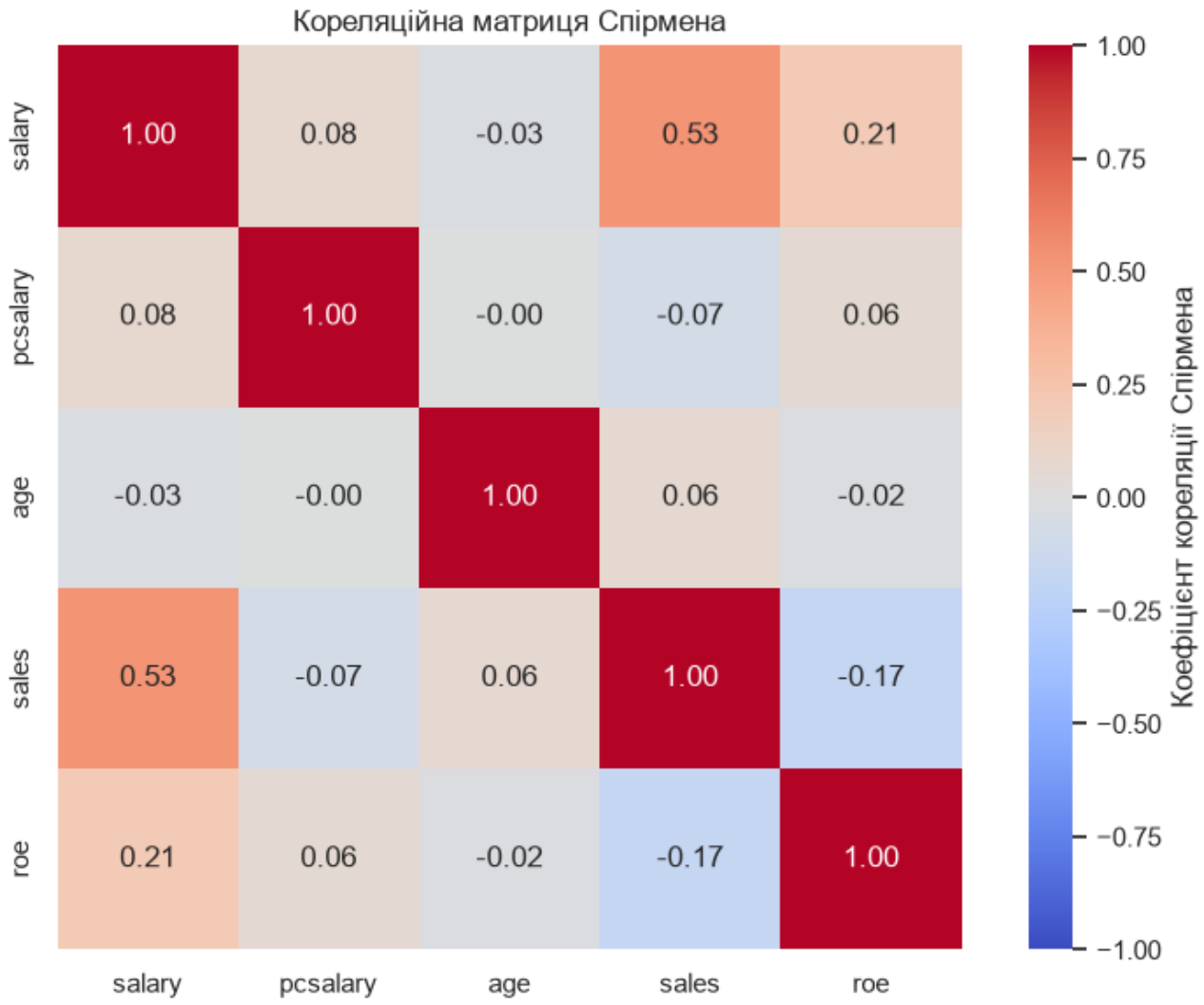

# --- Теплова карта Спірмена ---

plt.figure(figsize=(8, 6))

sns.heatmap(
    spearman_corr,
    annot=True,
    fmt=".2f",
    vmin=-1,
    vmax=1,
    center=0,
    cmap="coolwarm",
    square=True,
    cbar_kws={"label": "Коефіцієнт кореляції Спірмена"}
)

plt.title("Кореляційна матриця Спірмена")
plt.tight_layout()
plt.show()
```

	salary	pcsalary	age	sales	roe
salary	1.000	0.076	-0.030	0.528	0.206
pcsalary	0.076	1.000	-0.003	-0.074	0.058
age	-0.030	-0.003	1.000	0.063	-0.017
sales	0.528	-0.074	0.063	1.000	-0.171
roe	0.206	0.058	-0.017	-0.171	1.000




```
# Порівнюємо Pearson і Spearman для зв'язків із salary
salary_corr_comparison = pd.DataFrame({
    "Pearson": pearson_corr["salary"],
    "Spearman": spearman_corr["salary"]
})

display(salary_corr_comparison.round(3))
```

	Pearson	Spearman
salary	1.000	1.000
pcsalary	0.009	0.076
age	-0.110	-0.030
sales	0.120	0.528
roe	0.115	0.206

Інтерпретація

На `pairplot` видно, що для більшості пар змінних немає чіткої лінійної залежності. Крім того, у `salary` та `sales` є великі викиди і сильна правостороння асиметрія. Тому коефіцієнт Пірсона не завжди добре описує зв'язки в цих даних.

Найбільша різниця спостерігається для `salary` і `sales`:

- Пірсон: $r = 0.120$;
- Спірмен: $\rho_s = 0.528$.

За Пірсоном лінійний зв'язок виглядає дуже слабким, але Спірмен показує помітний позитивний монотонний зв'язок: компанії з більшими продажами загалом мають тенденцію платити CEO більше. Різниця виникає тому, що зв'язок не обов'язково є лінійним, а великі значення сильніше впливають на коефіцієнт Пірсона.

Для `salary` і `roe` також бачимо дещо більшу кореляцію Спірмена: 0.206 проти 0.115 у Пірсона, але зв'язок все одно залишається слабким.

Для `salary` і `pcsalary` обидва коефіцієнти близькі до нуля (0.009 і 0.076), тому помітного зв'язку немає.

Для `salary` і `age` обидва коефіцієнти також близькі до нуля (-0.110 і -0.030). При цьому `age` у цій роботі є змодельованою змінною, тому цей результат не слід трактувати як реальну економічну закономірність.

Отже, кореляція Спірмена виявляє деякі зв'язки, які Пірсон майже не показує, особливо між `salary` і `sales`. Для цих даних одного коефіцієнта Пірсона недостатньо, оскільки частина залежностей може бути монотонною, але не лінійною, а також присутні сильні викиди.

Частина 2(в)

Поясніть своїми словами концептуальну різницю між двома коефіцієнтами кореляції та пов'яжіть її з лінійною і монотонною залежністю.

Інтерпретація

Коефіцієнт кореляції **Пірсона** показує силу і напрямок **лінійного зв'язку** між двома змінними. Тобто він добре працює, коли залежність між змінними приблизно можна описати прямою лінією.

Коефіцієнт кореляції **Спірмена** працює з рангами значень і показує силу **монотонного зв'язку**. Монотонний зв'язок означає, що зі зростанням однієї змінної інша загалом або зростає, або спадає, але ця залежність не обов'язково має бути лінійною.

Тому Пірсон може показувати слабку кореляцію, якщо залежність є нелінійною. Спірмен у такому випадку може краще виявити загальну тенденцію.

У наших даних це добре видно на зв'язку між `salary` і `sales`: кореляція Пірсона дорівнює приблизно **0.12**, а Спірмена — **0.53**. Отже, між продажами компанії та зарплатою CEO є помітна позитивна монотонна тенденція, але вона погано описується прямою лінією.

Частина 2(г). Вікові підвибірки

Розгляньте дві підвибірки: CEO молодші за 50 років і старші за 50 років. Побудуйте для обох підвибірок накладені одна на одну гістограми / емпіричні функції розподілу (`ecdf`) та обговоріть результати. Чого ми можемо навчитися з відповідних мір положення та розсіювання?

```
# Формуємо дві підвибірки зарплат за віком CEO.
salary_under_50 = df.loc[df["age"] < 50, "salary"].dropna()
salary_over_50 = df.loc[df["age"] > 50, "salary"].dropna()

print(f"CEO молодші за 50 років: {len(salary_under_50)}")
print(f"CEO старші за 50 років: {len(salary_over_50)}")
print(f"CEO віком рівно 50 років: {(df['age'] == 50).sum()}")


# --- Міри положення та розсіювання ---

age_group_summary = pd.DataFrame({
    "Молодші за 50": [
        salary_under_50.mean(),
        salary_under_50.median(),
        salary_under_50.std(),
        salary_under_50.quantile(0.25),
        salary_under_50.quantile(0.75),
        salary_under_50.quantile(0.75) - salary_under_50.quantile(0.25)
    ],
    "Старші за 50": [
```

```
        salary_over_50.mean(),
        salary_over_50.median(),
        salary_over_50.std(),
        salary_over_50.quantile(0.25),
        salary_over_50.quantile(0.75),
        salary_over_50.quantile(0.75) - salary_over_50.quantile(0.25)
    ]
}, index=[
    "Середнє",
    "Медіана",
    "Стандартне відхилення",
    "Q1",
    "Q3",
    "IQR"
])

display(age_group_summary.round(2))
```

```
CEO молодші за 50 років: 66
CEO старші за 50 років: 143
CEO віком рівно 50 років: 0
```

	Молодші за 50	Старші за 50
Середнє	1488.14	1185.57
Медіана	1013.50	1045.00
Стандартне відхилення	2192.39	727.58
Q1	723.00	775.00
Q3	1332.50	1450.00
IQR	609.50	675.00

```
# Спільні межі інтервалів для обох підвибірок.
# Завдяки цьому стовпчики двох гістограм відповідають однаковим діапазонам зарплат.
age_group_bin_edges = np.histogram_bin_edges(
    pd.concat([salary_under_50, salary_over_50]),
    bins="auto"
)

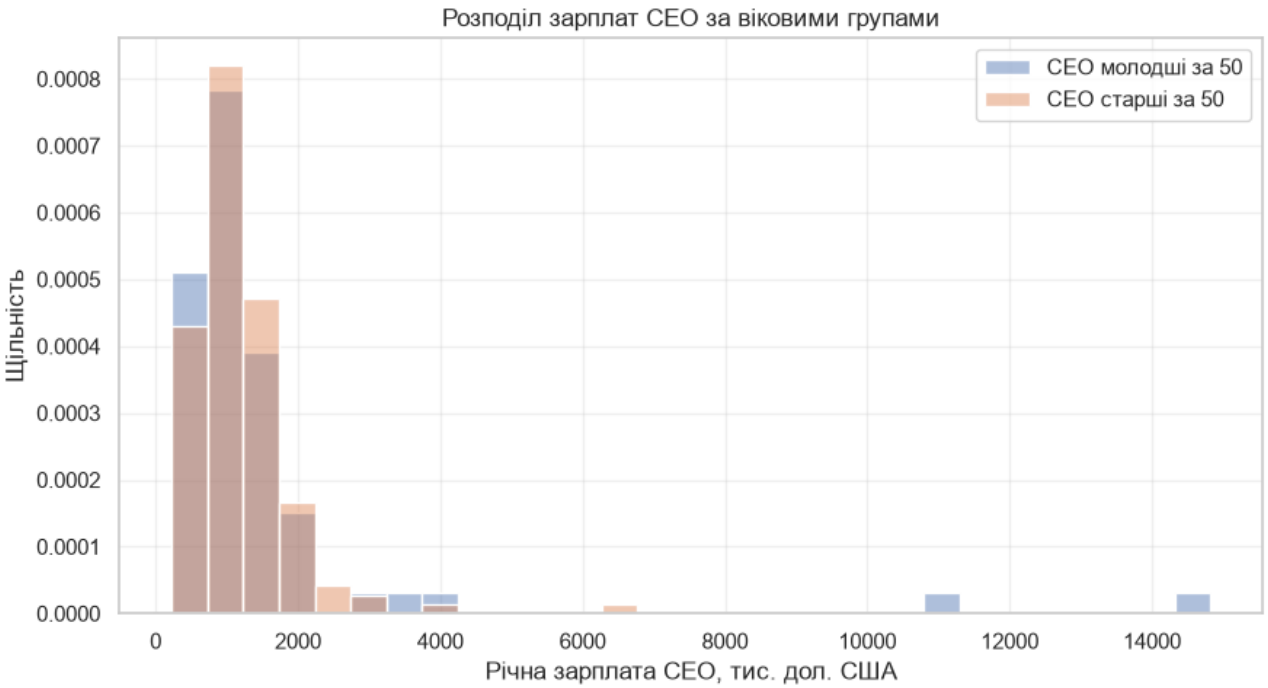
plt.figure(figsize=(9, 5))

sns.histplot(
    salary_under_50,
    bins=age_group_bin_edges,
    stat="density",
    alpha=0.45,
    label="CEO молодші за 50"
)

sns.histplot(
    salary_over_50,
    bins=age_group_bin_edges,
    stat="density",
    alpha=0.45,
    label="CEO старші за 50"
)

plt.title("Розподіл зарплат CEO за віковими групами")
plt.xlabel("Річна зарплата CEO, тис. дол. США")
plt.ylabel("Щільність")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



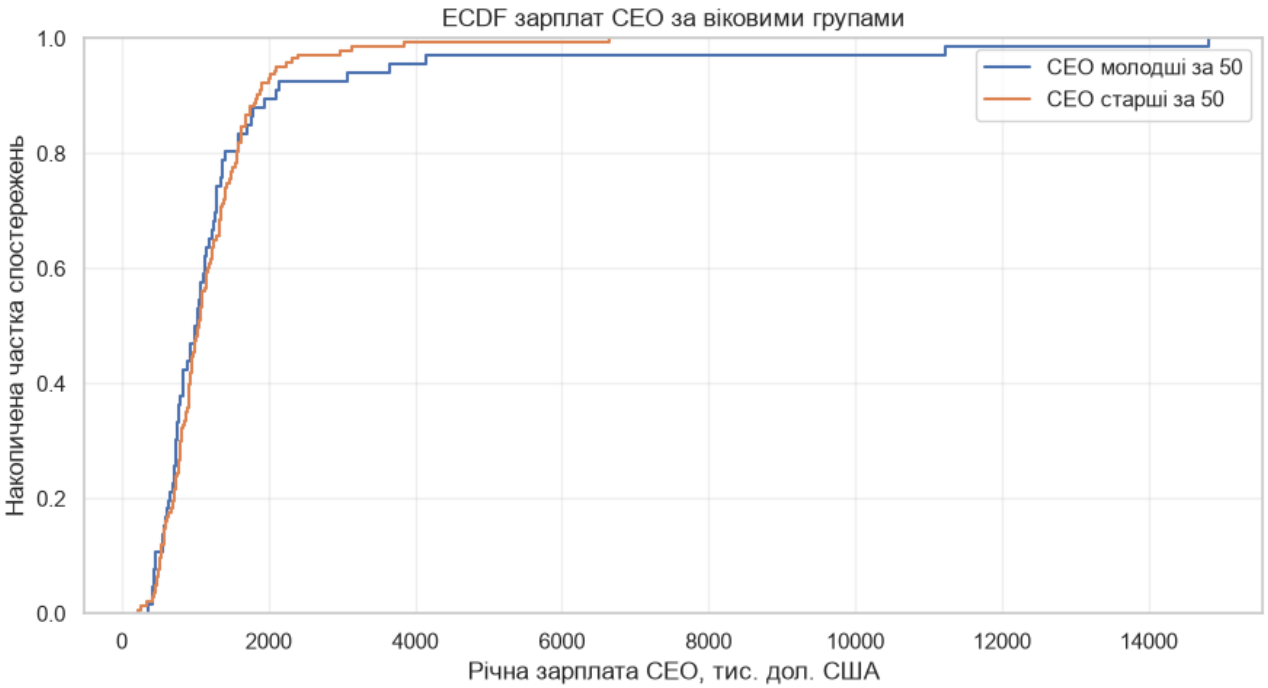

```
plt.figure(figsize=(9, 5))

sns.ecdfplot(
    salary_under_50,
    label="CEO молодші за 50"
)

sns.ecdfplot(
    salary_over_50,
    label="CEO старші за 50"
)

plt.title("ECDF зарплат CEO за віковими групами")
plt.xlabel("Річна зарплата CEO, тис. дол. США")
plt.ylabel("Накопичена частка спостережень")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Інтерпретація

У вибірці є 66 CEO молодших за 50 років і 143 CEO старших за 50 років. CEO віком рівно 50 років у вибірці немає.

Медіани зарплат у двох групах дуже близькі:

- молодші за 50: **1013.50 тис. дол. США**;
- старші за 50: **1045.00 тис. дол. США**.

Отже, типовий рівень зарплати в обох вікових групах є приблизно однаковим.

Середнє значення для молодших CEO є вищим:

- молодші за 50: **1488.14 тис. дол. США**;
- старші за 50: **1185.57 тис. дол. США**.

Але це пояснюється кількома дуже великими зарплатами у групі молодших CEO. Це також видно зі значно більшого стандартного відхилення: **2192.39** проти **727.58** тис. дол. США.

При цьому міжквартильний розмах досить схожий:

- молодші за 50: **IQR = 609.50**;
- старші за 50: **IQR = 675.00**.

Тобто центральні 50% зарплат у двох групах мають подібний розкид, а велика різниця у стандартному відхиленні виникає переважно через дуже високі окремі значення у групі молодших CEO.

Накладені гістограми та ECDF також показують, що основна частина двох розподілів сильно перекривається. Тому чіткої різниці у типовому рівні зарплати між цими двома віковими групами не видно.

Водночас `age` у цій роботі є штучно змодельованою змінною, тому ці результати не слід трактувати як реальний економічний висновок про вплив віку CEO на зарплату.

Частина 3(a). Таблиці спряженості

Визначте вікові групи A_1: `age < 55`, A_2: `age >= 55` та групи заробітної плати S_1: `salary < 700`, S_2: `700 <= salary < 1300`, S_3: `salary >= 1300`. Заробітну плату задано в тисячах доларів США.

За допомогою `pd.crosstab` побудуйте таблиці абсолютних та відносних частот розміром 2×3: рядки — A_1, A_2, стовпці — S_1, S_2, S_3. Відносні частоти обчислюйте від загальної кількості спостережень; додайте підсумки.

```
# --- Групи за рівнем зарплати ---

# right=False означає, що інтервали мають вигляд:
# [нижня межа, верхня межа)
# Тобто 700 потрапляє у S2, а 1300 — у S3.
salary_group = pd.cut(
    df["salary"],
    bins=[-np.inf, 700, 1300, np.inf],
    labels=["S1: <700", "S2: 700–1299", "S3: ≥1300"],
    right=False
)


# --- Групи за віком ---

# A1: age < 55
# A2: age >= 55
age_group = pd.cut(
    df["age"],
    bins=[-np.inf, 55, np.inf],
    labels=["A1: <55", "A2: ≥55"],
    right=False
)


# --- Абсолютні частоти ---

# Кожна комірка показує кількість CEO,
# які одночасно належать до відповідної вікової
# та зарплатної групи.
contingency_abs = pd.crosstab(
    age_group,
    salary_group,
    margins=True,
    margins_name="Разом"
)

print("Абсолютні частоти:")
display(contingency_abs)


# --- Відносні частоти ---

# normalize="all" ділить кожну комірку
# на загальну кількість спостережень.
# Тобто отримуємо частку від усієї вибірки.
contingency_rel = pd.crosstab(
    age_group,
    salary_group,
    normalize="all",
    margins=True,
    margins_name="Разом"
)

print("Відносні частоти:")
display(contingency_rel.round(3))
```

```
Абсолютні частоти:
```

salary age	S1: <700	S2: 700–1299	S3: ≥1300	Разом
A1: <55	27	66	44	137
A2: ≥55	15	35	22	72
Разом	42	101	66	209

```
Відносні частоти:
```

salary age	S1: <700	S2: 700–1299	S3: ≥1300	Разом
A1: <55	0.129	0.316	0.211	0.656
A2: ≥55	0.072	0.167	0.105	0.344
Разом	0.201	0.483	0.316	1.000

Частина 3(б). Інтерпретація частот

Дайте економічну та статистичну інтерпретацію n_{12}, h_{12}, $n_{1\cdot}$ та $h_{1\cdot}$ на основі таблиць.

Інтерпретація

У таблиці рядки відповідають віковим групам A_i, а стовпці — групам зарплати S_j.

- $n_{12} = 66$ — кількість CEO молодших за 55 років, які мають річну зарплату від 700 включно до 1300 невключно тис. дол. США.
- $h_{12} = 0.316$ — частка таких CEO у всій вибірці. Отже, приблизно 31.6% усіх спостережень належать одночасно до груп A_1 та S_2.
- $n_{1\cdot} = 137$ — загальна кількість CEO молодших за 55 років незалежно від рівня їхньої зарплати.
- $h_{1\cdot} = 0.656$ — частка CEO молодших за 55 років у всій вибірці, тобто приблизно 65.6%.

Частина 3(в). Залежність якісних ознак

Обчисліть відповідну міру залежності якісних ознак: коефіцієнт Крамера V або статистику критерію Пірсона χ^2. Поясніть результат і зробіть висновок про зв'язок віку та заробітної плати, враховуючи синтетичне походження віку.

```
# Беремо лише основну таблицю 2×3 без підсумкових
# рядка та стовпця "Разом".
contingency_core = contingency_abs.iloc[:-1, :-1]

# χ²-критерій Пірсона на незалежність.
chi2_stat, p_value, degrees_of_freedom, expected_freq = stats.chi2_contingency(
    contingency_core
)

print(f"χ² = {chi2_stat:.3f}")
print(f"p-value = {p_value:.4f}")
print(f"Ступені свободи = {degrees_of_freedom}")

# Очікувані частоти за умови незалежності
expected_table = pd.DataFrame(
    expected_freq,
    index=contingency_core.index,
    columns=contingency_core.columns
)

print("Очікувані частоти при незалежності:")
display(expected_table.round(2))
```

```
χ² = 0.068
p-value = 0.9666
Ступені свободи = 2
Очікувані частоти при незалежності:
```

salary age	S1: <700	S2: 700–1299	S3: ≥1300
A1: <55	27.53	66.21	43.26
A2: ≥55	14.47	34.79	22.74

```
# Розміри таблиці
rows, cols = contingency_core.shape

# Коефіцієнт Крамера V:
# показує силу зв'язку між двома категоріальними ознаками.
cramers_v = np.sqrt(
    chi2_stat
    / (contingency_core.to_numpy().sum() * min(rows - 1, cols - 1))
)

print(f"Коефіцієнт Крамера V = {cramers_v:.3f}")
```

```
Коефіцієнт Крамера V = 0.018
```

Інтерпретація

Для перевірки зв’язку між віковою групою CEO та категорією зарплати використано χ^2-критерій незалежності Пірсона.

Отримано:

$$\chi^2 = 0.068, \qquad p\text{-value} = 0.9666.$$

Оскільки p-value > 0.05, немає підстав вважати, що вікова група та категорія зарплати пов’язані між собою.

Очікувані частоти при незалежності дуже близькі до фактичних частот у таблиці, що також вказує на відсутність помітної залежності.

Коефіцієнт Крамера становить:

$$V = 0.018.$$

Це значення дуже близьке до нуля, тому сила зв’язку між віковою групою та рівнем зарплати є практично відсутньою.

Важливо, що змінна `age` у цій роботі була синтетично згенерована, тому цей результат не слід трактувати як реальний економічний висновок про вплив віку CEO на його зарплату.

Задача 2. Змодельовані дані

Нумерацію збережено відповідно до умови: після частини 1 йде частина 3. Для відтворюваності моделювання використовується `SEED = 42`.

Для відтворюваності обчислень використано Python та бібліотеки NumPy, pandas, SciPy, matplotlib і seaborn. Початкове значення генератора випадкових чисел: `SEED = 42`.

Частина 1(a). Забруднення вибірки

Згенеруйте базову вибірку обсягом $n_1 = 100$ із розподілу $N(10, 1^2)$. Додайте до неї m спостережень із розподілу $N(20, 2^2)$, тобто із середнім 20 та стандартним відхиленням 2 (дисперсією 4).

Змінюйте m від 10 до 200 з кроком 20. **Уточнення межі:** такий крок дає значення $10, 30, \ldots, 190$; 200 до цієї послідовності не входить. Використайте цю послідовність, а якщо додаєте 200 як окрему кінцеву точку, явно зазначте це.

Для об'єднаної вибірки обчисліть і зобразіть середнє, медіану та вибіркову дисперсію як функції m. Зазначте використаний знаменник у формулі дисперсії. Обговоріть вплив зростання неоднорідності на ці міри положення й розсіювання.

```
# Окремий генератор для цієї частини,
# щоб результат не залежав від запуску інших випадкових блоків notebook.
rng_contamination = np.random.default_rng(SEED)

# --- Базова вибірка ---

# 100 спостережень з N(10, 1²)
base_sample = rng_contamination.normal(
    loc=10,      # середнє μ₁
    scale=1,     # стандартне відхилення σ₁
    size=100
)


# --- Забруднююча вибірка ---

# Генеруємо одразу 200 спостережень з N(20, 2²).
# Для кожного m нижче будемо брати перші m значень.
contamination_sample = rng_contamination.normal(
    loc=20,      # середнє μ₂
    scale=2,     # стандартне відхилення σ₂
    size=200
)


# За умовою: починаємо з m=10 і збільшуємо на 20.
# Така послідовність дає 10, 30, ..., 190.
m_values = np.arange(10, 200, 20)

results_contamination = []

for m in m_values:
    # Беремо перші m спостережень забруднюючої вибірки
    contamination = contamination_sample[:m]

    # Об'єднуємо базову і забруднюючу вибірки
    combined_sample = np.concatenate([
        base_sample,
        contamination
    ])

    # Обчислюємо характеристики об'єднаної вибірки
    results_contamination.append({
        "m": m,
        "Середнє": np.mean(combined_sample),
```

```
        "Медіана": np.median(combined_sample),

        # ddof=1 → вибіркова дисперсія з дільником n-1
        "Вибіркова дисперсія": np.var(combined_sample, ddof=1)
    })


# Перетворюємо результати у таблицю
contamination_results = pd.DataFrame(results_contamination)

display(contamination_results.round(3))
```

	m	Середнє	Медіана	Вибіркова дисперсія
0	10	10.827	10.135	8.594
1	30	12.245	10.384	18.846
2	50	13.266	10.538	23.865
3	70	14.040	10.727	26.012
4	90	14.692	11.048	27.277
5	110	15.176	17.083	27.243
6	130	15.627	17.708	27.313
7	150	15.923	17.902	26.702
8	170	16.232	18.170	26.052
9	190	16.503	18.366	25.474

Інтерпретація / виведення

Зі збільшенням кількості спостережень m з розподілу $N(20, 2^2)$ середнє значення об'єднаної вибірки поступово зростає: від **10.827** при $m = 10$ до **16.503** при $m = 190$. Це логічно, оскільки до початкової вибірки із середнім близько 10 ми додаємо все більше значень із середнім близько 20.

Медіана спочатку змінюється значно повільніше: при $m = 10$ вона дорівнює **10.135**, а при $m = 90$ — лише **11.048**. Це показує, що медіана стійкіша до додавання меншої кількості сильно відмінних спостережень.

Коли m стає більшим за 100, друга група вже становить більше половини об'єднаної вибірки, тому медіана різко переходить ближче до її центру: при $m = 110$ вона вже дорівнює **17.083**.

Вибіркова дисперсія зростає приблизно від **8.594** до значень близько **27**. Причина в тому, що об'єднуються дві групи з дуже різними середніми — близько 10 і 20, тому загальний розкид даних сильно збільшується.

Найбільша дисперсія спостерігається, коли обидві групи мають приблизно порівнянні розміри. Після цього вона трохи зменшується, оскільки друга група починає переважати і об'єднана вибірка знову стає більш однорідною.

Частина 1(б). Графіки розподілів

Для кожного $m \in \{20, 100, 200\}$ побудуйте діаграми розмаху та гістограми щільності:

1. для базової підвибірки з $N(10, 1^2)$;
2. для доданої підвибірки з $N(20, 2^2)$;
3. для всієї об'єднаної вибірки.

Порівняйте форму розподілів та їх зміну зі збільшенням m. Ці три значення m задані окремо від послідовності в пункті (а).

```
# Контрастні значення m, які прямо вказані в умові
contrast_m_values = [20, 100, 200]

for m in contrast_m_values:
    # Беремо перші m спостережень із уже згенерованої забруднюючої вибірки
    contamination = contamination_sample[:m]
```

```
# Об'єднуємо базову та забруднюючу вибірки
combined_sample = np.concatenate([
    base_sample,
    contamination
])

# Використовуємо однакові межі інтервалів для всіх трьох гістограм,
# щоб їх можна було коректно порівнювати між собою.
common_bin_edges = np.histogram_bin_edges(
    combined_sample,
    bins="auto"
)

# Спільні межі осі X для всіх графіків
x_min = combined_sample.min()
x_max = combined_sample.max()

fig, axes = plt.subplots(
    2,
    3,
    figsize=(15, 7)
)

# --- Гістограма базової вибірки ---
sns.histplot(
    base_sample,
    bins=common_bin_edges,
    stat="density",
    edgecolor="black",
    ax=axes[0, 0]
)
axes[0, 0].set_title("Базова вибірка N(10, 1²)")
axes[0, 0].set_ylabel("Щільність")
axes[0, 0].set_xlim(x_min, x_max)
axes[0, 0].grid(True, alpha=0.3)

# --- Гістограма забруднюючої вибірки ---
sns.histplot(
    contamination,
    bins=common_bin_edges,
    stat="density",
    edgecolor="black",
    ax=axes[0, 1]
)
axes[0, 1].set_title(f"Забруднення N(20, 2²), m={m}")
axes[0, 1].set_ylabel("Щільність")
axes[0, 1].set_xlim(x_min, x_max)
axes[0, 1].grid(True, alpha=0.3)

# --- Гістограма об'єднаної вибірки ---
sns.histplot(
    combined_sample,
    bins=common_bin_edges,
    stat="density",
    edgecolor="black",
    ax=axes[0, 2]
)
axes[0, 2].set_title("Об'єднана вибірка")
axes[0, 2].set_ylabel("Щільність")
axes[0, 2].set_xlim(x_min, x_max)
axes[0, 2].grid(True, alpha=0.3)

# --- Boxplot базової вибірки ---
sns.boxplot(
    x=base_sample,
    ax=axes[1, 0]
```

```
)
axes[1, 0].set_title("Boxplot базової вибірки")
axes[1, 0].set_xlim(x_min, x_max)
axes[1, 0].grid(True, axis="x", alpha=0.3)

# --- Boxplot забруднюючої вибірки ---
sns.boxplot(
    x=contamination,
    ax=axes[1, 1]
)
axes[1, 1].set_title(f"Boxplot забруднення, m={m}")
axes[1, 1].set_xlim(x_min, x_max)
axes[1, 1].grid(True, axis="x", alpha=0.3)

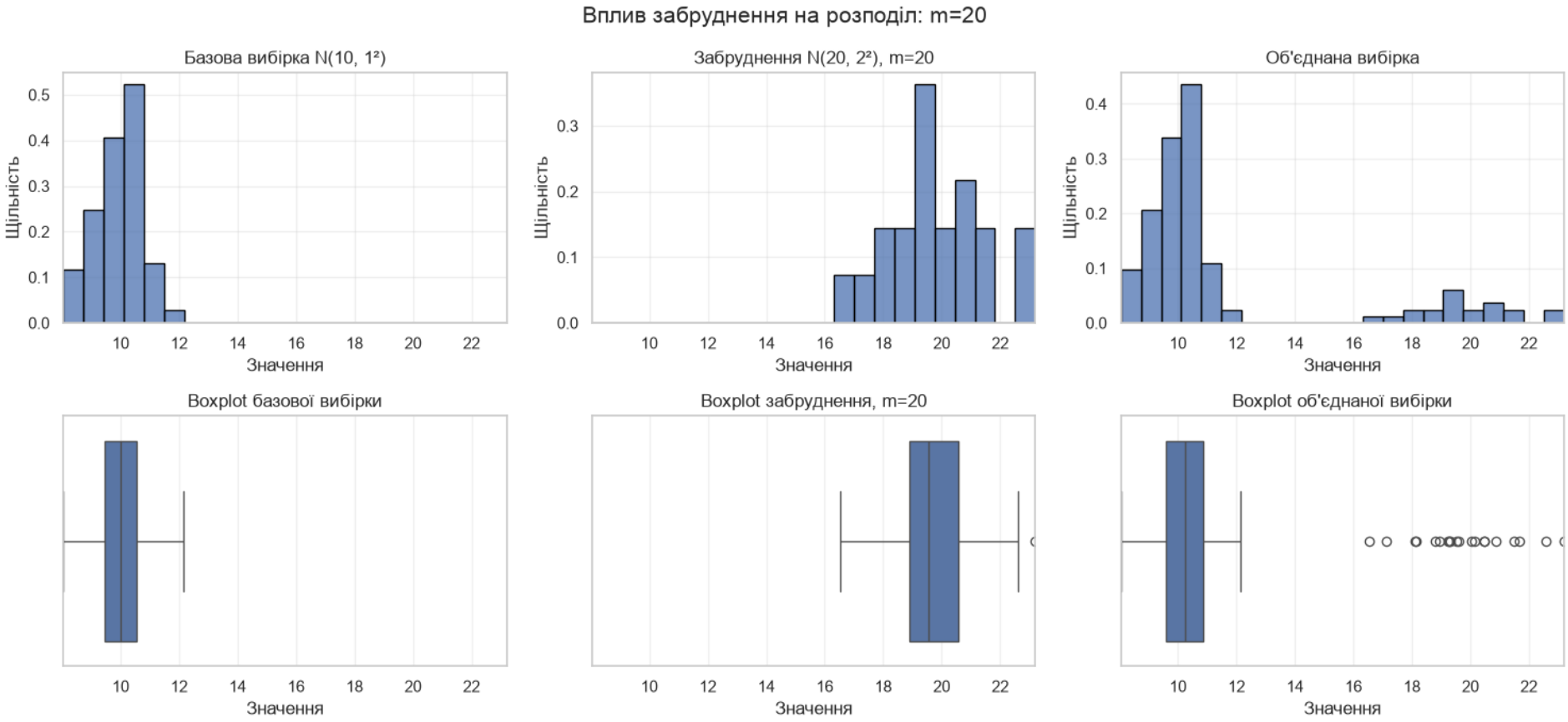
# --- Boxplot об'єднаної вибірки ---
sns.boxplot(
    x=combined_sample,
    ax=axes[1, 2]
)
axes[1, 2].set_title("Boxplot об'єднаної вибірки")
axes[1, 2].set_xlim(x_min, x_max)
axes[1, 2].grid(True, axis="x", alpha=0.3)

# Підпис осі X
for ax in axes.flat:
    ax.set_xlabel("Значення")

fig.suptitle(
    f"Вплив забруднення на розподіл: m={m}",
    fontsize=15
)

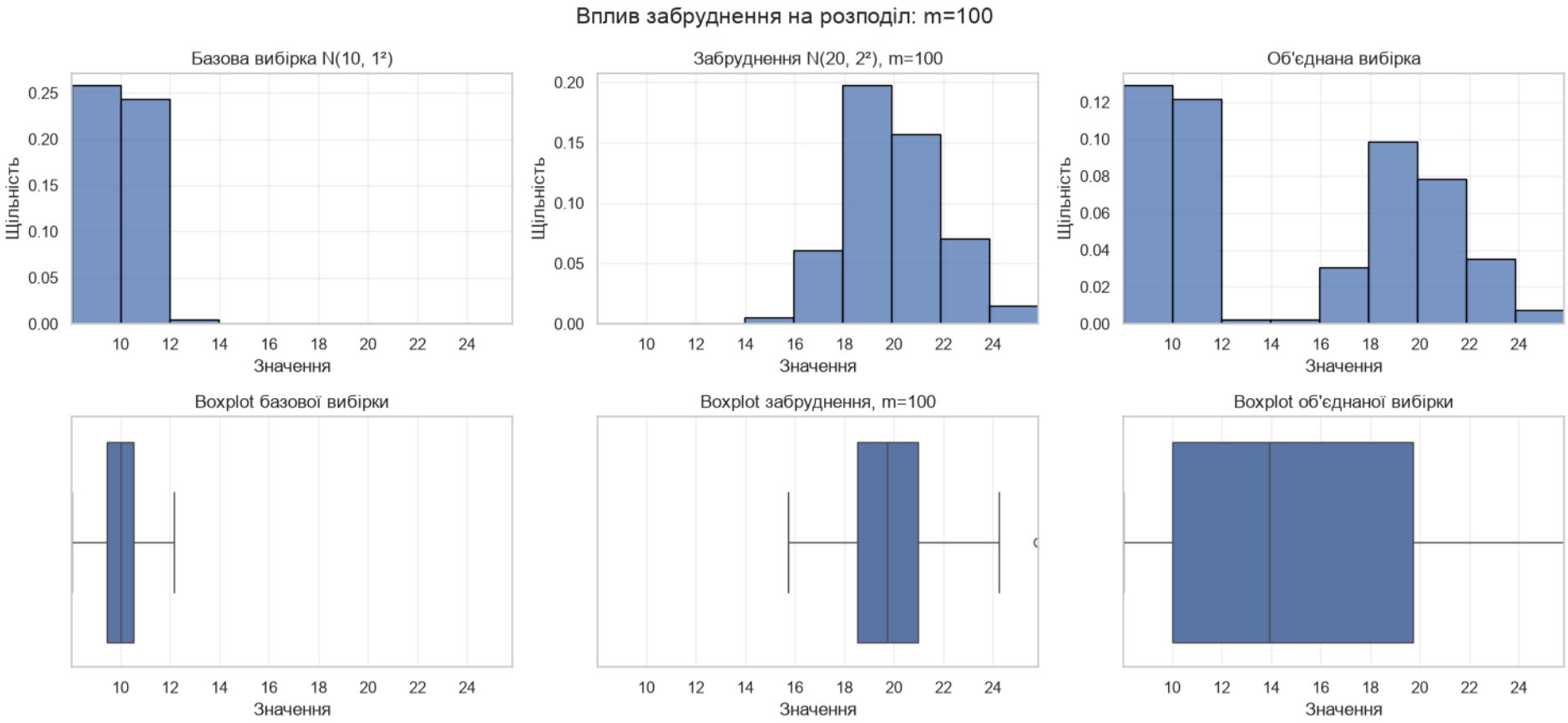
plt.tight_layout()
plt.show()
```

Розподіли базової, доданої та об’єднаної вибірок



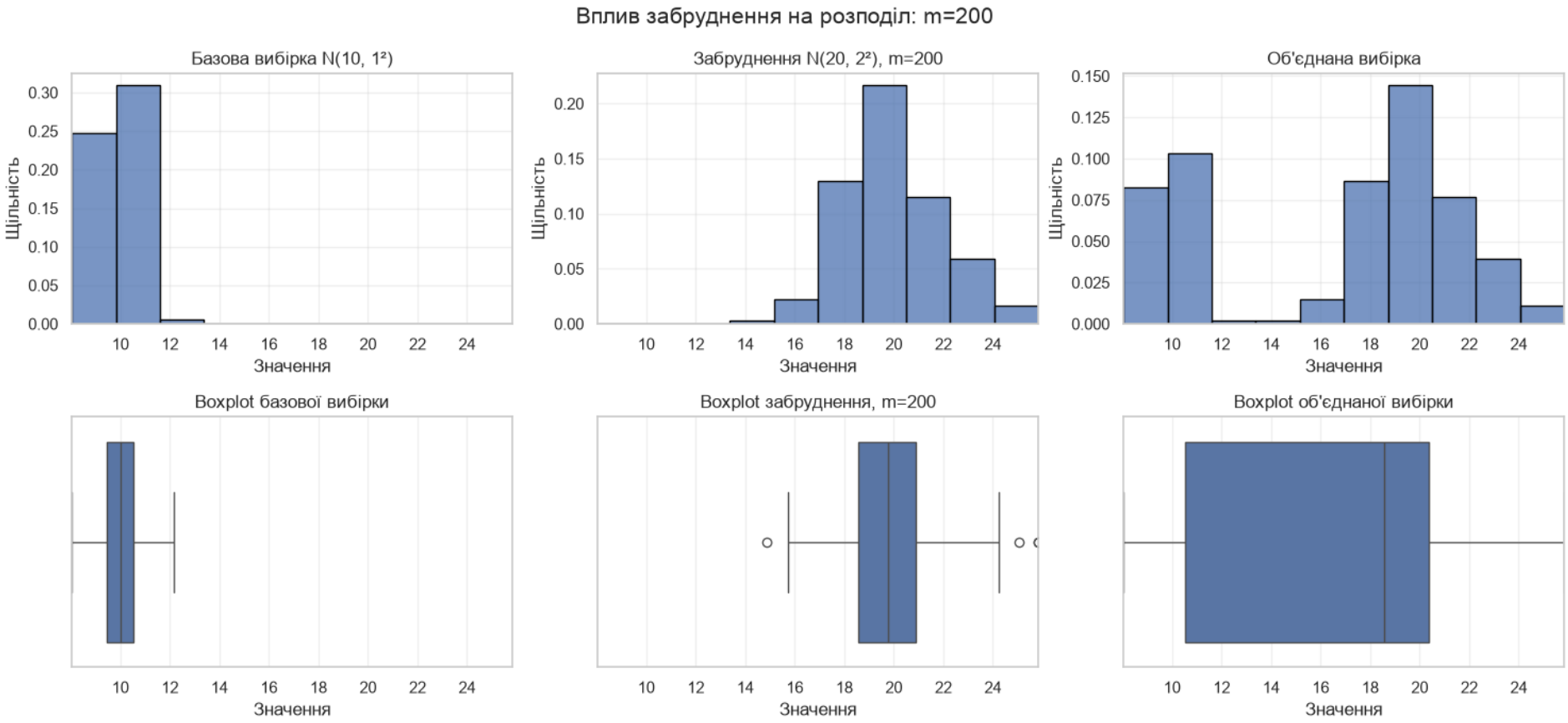
Значення змінної є змодельованими; гістограми показують щільність розподілу.

Розподіли базової, доданої та об’єднаної вибірок



Значення змінної є змодельованими; гістограми показують щільність розподілу.

Розподіли базової, доданої та об’єднаної вибірок



Значення змінної є змодельованими; гістограми показують щільність розподілу.

Інтерпретація

Базова вибірка з $N(10, 1^2)$ зосереджена біля 10, а забруднююча вибірка з $N(20, 2^2)$ — біля 20 і має більший розкид.

При $m = 20$ більшість об'єднаної вибірки все ще складають спостереження з базової групи. На гістограмі видно основний пік біля 10 і невелику окрему групу біля 20. На boxplot значення другої групи переважно позначаються як викиди. У цьому випадку це не випадкові помилки, а спостереження з іншої підвибірки.

При $m = 100$ обидві групи мають однаковий розмір. На гістограмі добре видно два окремі скупчення біля 10 і 20. Об'єднаний розподіл є неоднорідним і має два виражені піки.

При $m = 200$ друга група вже переважає, тому основна частина об'єднаної вибірки зміщується в напрямку значень біля 20. Водночас група біля 10 все ще добре помітна на гістограмі.

Ці графіки показують, що зі збільшенням m змінюється не лише середнє чи медіана, а й сама форма розподілу. Також видно обмеження boxplot: він добре показує положення та розкид, але може приховувати наявність двох окремих груп, яку значно краще видно на гістограмі.

Частина 3(a). Теоретичне обґрунтування

Нехай $U, V \sim N(0, 1)$ — незалежні випадкові величини. Для $\rho \in [-1, 1]$ визначте

$$U^* = U, \qquad V^* = \rho U + \sqrt{1 - \rho^2}\, V.$$

Математично доведіть, що $\text{Var}(U^*) = \text{Var}(V^*) = 1$ та $\text{Corr}(U^*, V^*) = \rho$. Наведіть виведення у текстовій клітинці нижче.

Відповідь

Нехай $U, V \sim N(0, 1)$ — незалежні випадкові величини. Тоді

$$E[U] = E[V] = 0, \qquad \text{Var}(U) = \text{Var}(V) = 1.$$

Оскільки U і V незалежні, їхня **коваріація (covariance)** дорівнює нулю:

$$\text{Cov}(U, V) = 0.$$

Визначимо

$$U^* = U, \qquad V^* = \rho U + \sqrt{1 - \rho^2} V,$$

де $\rho \in [-1, 1]$ — задане значення кореляції.

Для U^*:

$$\text{Var}(U^*) = \text{Var}(U) = 1.$$

Для V^* використаємо властивість дисперсії лінійної комбінації:

$$\text{Var}(aU + bV) = a^2\, \text{Var}(U) + b^2\, \text{Var}(V) + 2ab\, \text{Cov}(U, V).$$

У нашому випадку $a = \rho$, $b = \sqrt{1 - \rho^2}$, а $\text{Cov}(U, V) = 0$, тому

$$\text{Var}(V^*) = \rho^2 \cdot 1 + (1 - \rho^2) \cdot 1 = 1.$$

Отже,

$$\boxed{\mathrm{Var}(U^*) = \mathrm{Var}(V^*) = 1}.$$

Тепер знайдемо кореляцію. За означенням:

$$\mathrm{Corr}(U^*, V^*) = \frac{\mathrm{Cov}(U^*, V^*)}{\sqrt{\mathrm{Var}(U^*)\,\mathrm{Var}(V^*)}}.$$

Оскільки обидві дисперсії дорівнюють 1, достатньо знайти коваріацію:

$$\mathrm{Cov}(U^*, V^*) = \mathrm{Cov}\left(U, \rho U + \sqrt{1-\rho^2}V\right).$$

Використовуючи лінійність коваріації:

$$= \rho\,\mathrm{Cov}(U, U) + \sqrt{1-\rho^2}\,\mathrm{Cov}(U, V).$$

При цьому

$$\mathrm{Cov}(U, U) = \mathrm{Var}(U) = 1, \qquad \mathrm{Cov}(U, V) = 0.$$

Тому

$$\mathrm{Cov}(U^*, V^*) = \rho.$$

Отже,

$$\boxed{\mathrm{Corr}(U^*, V^*) = \rho}.$$

Таким чином, ця конструкція дозволяє отримати дві випадкові величини з дисперсіями 1 і наперед заданою кореляцією ρ.

Частина 3(б). Моделювання кореляції

За наведеним алгоритмом згенеруйте дві вибірки обсягом $n = 100$ для кожного ρ від -0.9 до 0.9 з кроком 0.2. Обчисліть вибіркові коефіцієнти кореляції Пірсона r та Спірмена ρ_{sp}. Побудуйте графік їх залежності від заданого теоретичного параметра ρ та порівняйте результати двох методів.

```
# Окремий генератор випадкових чисел для цієї частини,
# щоб результат був відтворюваним і не залежав від інших блоків notebook.
rng_correlation = np.random.default_rng(SEED)

# Розмір кожної вибірки
correlation_sample_size = 100

# Значення теоретичної кореляції rho:
# -0.9, -0.7, ..., 0.7, 0.9
rho_values = np.round(np.arange(-0.9, 1.0, 0.2), 1)

correlation_results = []

for rho in rho_values:
    # Генеруємо дві незалежні стандартні нормальні вибірки
    U = rng_correlation.normal(loc=0, scale=1, size=correlation_sample_size)
    V = rng_correlation.normal(loc=0, scale=1, size=correlation_sample_size)

    # Будуємо нові змінні за формулою з попереднього підпункту
    U_star = U
    V_star = rho * U + np.sqrt(1 - rho**2) * V
```

```
    # Вибіркова кореляція Пірсона
    pearson_r = stats.pearsonr(U_star, V_star).statistic

    # Вибіркова кореляція Спірмена
    spearman_rho = stats.spearmanr(U_star, V_star).statistic

    correlation_results.append({
        "Теоретичне ρ": rho,
        "Пірсон": pearson_r,
        "Спірмен": spearman_rho
    })

# Таблиця результатів
correlation_simulation = pd.DataFrame(correlation_results)

display(correlation_simulation.round(3))
```

	Теоретичне ρ	Пірсон	Спірмен
0	-0.9	-0.840	-0.832
1	-0.7	-0.717	-0.635
2	-0.5	-0.438	-0.455
3	-0.3	-0.392	-0.341
4	-0.1	-0.112	-0.102
5	0.1	-0.120	-0.117
6	0.3	0.270	0.264
7	0.5	0.492	0.466
8	0.7	0.702	0.711
9	0.9	0.899	0.894

```
plt.figure(figsize=(9, 5))

# Теоретична кореляція: ідеальна лінія y = rho
plt.plot(
    correlation_simulation["Теоретичне ρ"],
    correlation_simulation["Теоретичне ρ"],
    linestyle="--",
    marker="o",
    label="Теоретичне ρ"
)

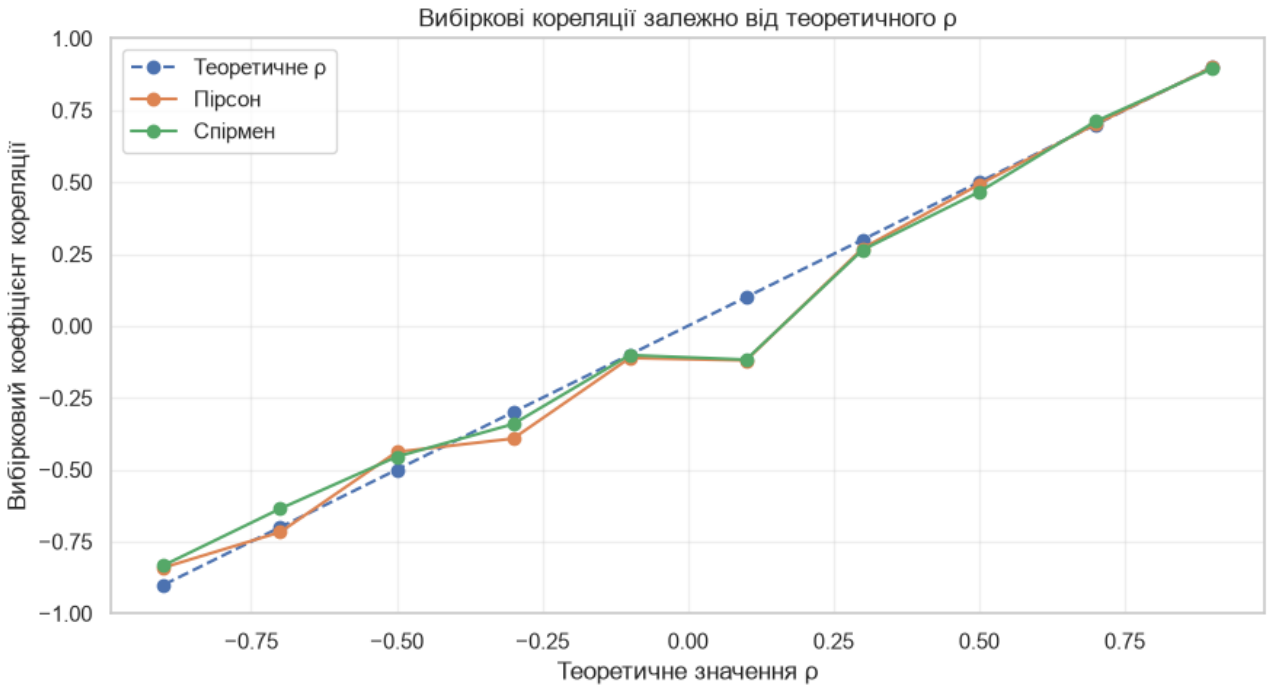
# Вибіркова кореляція Пірсона
plt.plot(
    correlation_simulation["Теоретичне ρ"],
    correlation_simulation["Пірсон"],
    marker="o",
    label="Пірсон"
)

# Вибіркова кореляція Спірмена
plt.plot(
    correlation_simulation["Теоретичне ρ"],
    correlation_simulation["Спірмен"],
    marker="o",
    label="Спірмен"
)

plt.title("Вибіркові кореляції залежно від теоретичного ρ")
plt.xlabel("Теоретичне значення ρ")
plt.ylabel("Вибірковий коефіцієнт кореляції")
plt.ylim(-1, 1)
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
```

```
plt.show()
```



Інтерпретація

Вибіркова кореляція Пірсона в цілому добре наближає задане теоретичне значення ρ. Чим сильніша за модулем задана кореляція, тим чіткіше вибіркове значення повторює її напрямок і величину.

Точного збігу з ρ немає, оскільки використовується випадкова вибірка лише з $n = 100$ спостережень. Наприклад, при $\rho = 0.1$ вибіркова кореляція Пірсона дорівнює -0.120. Оскільки справжній зв'язок тут дуже слабкий, випадкові коливання вибірки можуть навіть тимчасово змінити його знак.

Кореляції Пірсона та Спірмена в більшості випадків мають близькі значення. Це очікувано, оскільки між U^* та V^* задається лінійний і водночас монотонний зв'язок.

Пірсон безпосередньо оцінює лінійну кореляцію, тому саме його значення повинні наближатися до заданого параметра ρ. Спірмен працює з рангами, тому його значення не зобов'язані точно дорівнювати ρ, але в цьому експерименті вони поводяться дуже подібно.

Отже, моделювання підтверджує теоретичний результат: заданий параметр ρ визначає напрямок і силу лінійного зв'язку між U^* та V^*, а вибіркові коефіцієнти наближають цей зв'язок із певною випадковою похибкою.

Частина 3(в). Монотонне перетворення

Виконайте нелінійне, але строго монотонне перетворення $V_{\text{exp}} = \exp(V^*)$. Порівняйте коефіцієнти Пірсона та Спірмена для пар (U^*, V^*) і (U^*, V_{exp}), використовуючи ті самі змодельовані спостереження. Візуалізуйте результати та обґрунтуйте економіко-статистичні висновки.

```
# Створюємо окремий генератор з тим самим SEED.
# Оскільки порядок генерації такий самий, отримаємо ті самі
# змодельовані спостереження, що й у Частині 3(б).
rng_monotonic = np.random.default_rng(SEED)

monotonic_results = []

for rho in rho_values:
    # Ті самі незалежні стандартні нормальні вибірки
    U_mono = rng_monotonic.normal(
        loc=0,
```

```
        scale=1,
        size=correlation_sample_size
    )

    V_mono = rng_monotonic.normal(
        loc=0,
        scale=1,
        size=correlation_sample_size
    )

    # Початкові змінні з заданою кореляцією rho
    U_star_mono = U_mono
    V_star_mono = (
        rho * U_mono
        + np.sqrt(1 - rho**2) * V_mono
    )

    # Нелінійне, але строго зростаюче перетворення
    V_exp = np.exp(V_star_mono)

    # Кореляції для початкової пари
    pearson_original = stats.pearsonr(
        U_star_mono,
        V_star_mono
    ).statistic

    spearman_original = stats.spearmanr(
        U_star_mono,
        V_star_mono
    ).statistic

    # Кореляції після exp(V*)
    pearson_exp = stats.pearsonr(
        U_star_mono,
        V_exp
    ).statistic

    spearman_exp = stats.spearmanr(
        U_star_mono,
        V_exp
    ).statistic

    monotonic_results.append({
        "Теоретичне ρ": rho,
        "Пірсон до exp": pearson_original,
        "Пірсон після exp": pearson_exp,
        "Спірмен до exp": spearman_original,
        "Спірмен після exp": spearman_exp
    })

monotonic_comparison = pd.DataFrame(monotonic_results)

display(monotonic_comparison.round(3))
```

	Теоретичне ρ	Пірсон до exp	Пірсон після exp	Спірмен до exp	Спірмен після exp
0	-0.9	-0.840	-0.777	-0.832	-0.832
1	-0.7	-0.717	-0.624	-0.635	-0.635
2	-0.5	-0.438	-0.356	-0.455	-0.455
3	-0.3	-0.392	-0.387	-0.341	-0.341
4	-0.1	-0.112	-0.006	-0.102	-0.102
5	0.1	-0.120	-0.109	-0.117	-0.117
6	0.3	0.270	0.203	0.264	0.264
7	0.5	0.492	0.409	0.466	0.466
8	0.7	0.702	0.548	0.711	0.711
9	0.9	0.899	0.741	0.894	0.894

```
plt.figure(figsize=(10, 6))

plt.plot(
    monotonic_comparison["Теоретичне ρ"],
    monotonic_comparison["Пірсон до exp"],
    marker="o",
    label="Пірсон: (U*, V*)"
)

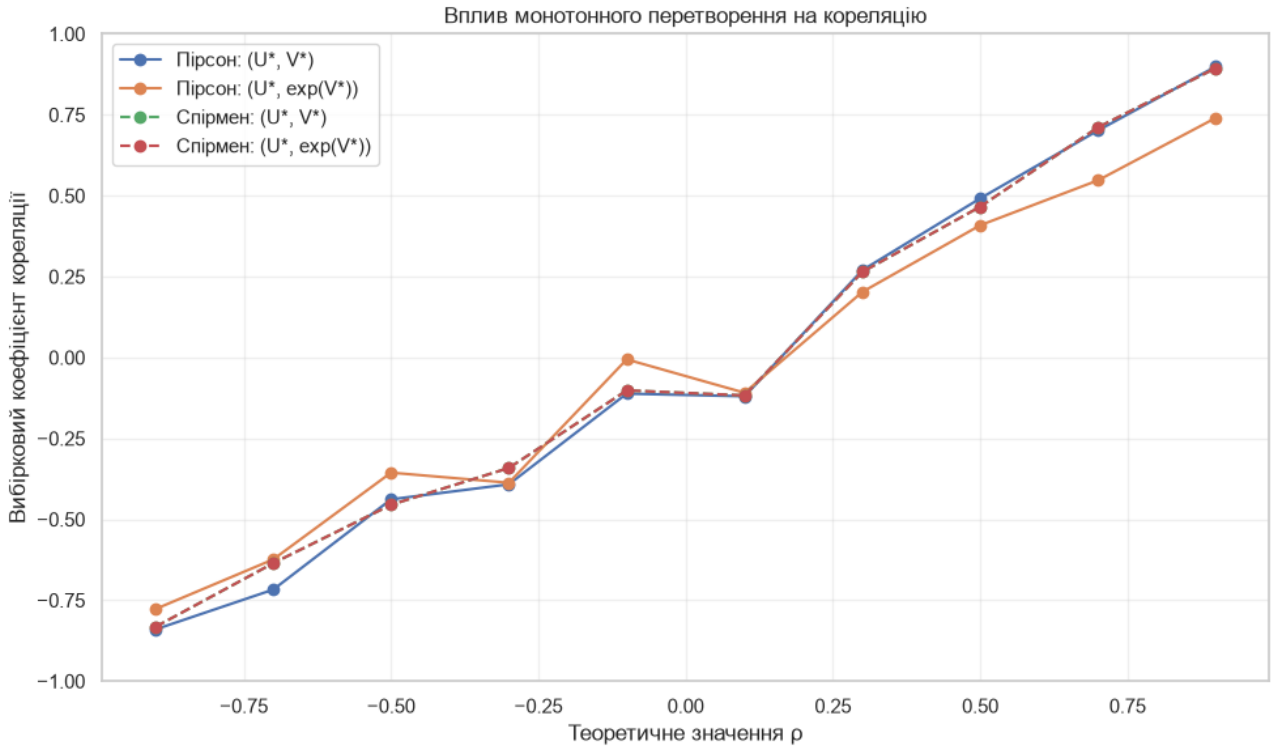
plt.plot(
    monotonic_comparison["Теоретичне ρ"],
    monotonic_comparison["Пірсон після exp"],
    marker="o",
    label="Пірсон: (U*, exp(V*))"
)

plt.plot(
    monotonic_comparison["Теоретичне ρ"],
    monotonic_comparison["Спірмен до exp"],
    marker="o",
    linestyle="--",
    label="Спірмен: (U*, V*)"
)

plt.plot(
    monotonic_comparison["Теоретичне ρ"],
    monotonic_comparison["Спірмен після exp"],
    marker="o",
    linestyle="--",
    label="Спірмен: (U*, exp(V*))"
)

plt.title("Вплив монотонного перетворення на кореляцію")
plt.xlabel("Теоретичне значення ρ")
plt.ylabel("Вибірковий коефіцієнт кореляції")
plt.ylim(-1, 1)
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Інтерпретація

Після перетворення $V_{\text{exp}} = \exp(V^*)$ коефіцієнт Спірмена не змінився для жодного значення ρ. Це відбувається тому, що експонента є строго зростаючою функцією: вона змінює самі значення, але не змінює їх порядок. Наприклад, при $\rho = 0.9$ кореляція Спірмена до і після перетворення дорівнює **0.894**.

Коефіцієнт Пірсона після експоненційного перетворення змінився. Наприклад, при $\rho = 0.9$ він зменшився з **0.899** до **0.741**, а при $\rho = -0.1$ — з **-0.112** до **-0.006**. Причина в тому, що експонента робить залежність нелінійною, а Пірсон вимірює саме силу лінійного зв'язку.

Отже, кореляція Спірмена є стійкою до строго монотонних перетворень, оскільки працює з рангами. Кореляція Пірсона залежить від конкретної форми зв'язку між числовими значеннями і тому може змінюватися після нелінійного перетворення.

З практичної точки зору, якщо між економічними показниками є стабільна тенденція зростання або спадання, але вона не є лінійною, Спірмен може краще відобразити цей зв'язок, ніж Пірсон.

Задача 3. Ймовірності та розподіли

Для відтворюваності обчислень використано Python та бібліотеки NumPy, pandas, SciPy, matplotlib і seaborn. Початкове значення генератора випадкових чисел: `SEED = 42`.

Частина 1. Результати іспиту

Усі студенти належать до однієї з двох груп: A — добре підготовлені до іспиту, B — не дуже добре підготовлені. Нехай E — подія «випадково вибраний студент складає іспит». Відомо:

$$P(B) = 0.1, \qquad P(E \mid A) = 0.8, \qquad P(E \mid B) = 0.3.$$

- **(а)** Знайдіть імовірність того, що студент складе іспит **і** належить до групи A: $P(E \cap A)$.
- **(б)** Знайдіть імовірність скласти іспит: $P(E)$.
- **(в)** Знайдіть імовірність належати до групи B за умови, що студент склав іспит: $P(B \mid E)$.

Наведіть формули та поясніть результати.

```
# Відомі ймовірності
P_B = 0.1
P_E_given_A = 0.8
P_E_given_B = 0.3

# Оскільки студент належить або до A, або до B
P_A = 1 - P_B

# (а) Ймовірність одночасно належати до A і скласти іспит
P_E_and_A = P_E_given_A * P_A

# (б) Загальна ймовірність скласти іспит:
# формула повної ймовірності
P_E = P_E_given_A * P_A + P_E_given_B * P_B

# (в) Ймовірність належати до B за умови, що студент склав іспит:
# формула Байєса
P_B_given_E = (P_E_given_B * P_B) / P_E

print(f"P(A) = {P_A:.2f}")
print(f"P(E ∩ A) = {P_E_and_A:.2f}")
print(f"P(E) = {P_E:.2f}")
print(f"P(B | E) = {P_B_given_E:.2f}")
```

```
P(A) = 0.90
P(E ∩ A) = 0.72
P(E) = 0.75
P(B | E) = 0.04
```

Інтерпретація / виведення

Оскільки $P(B) = 0.1$, маємо $P(A) = 0.9$.

(а) Ймовірність того, що студент одночасно належить до групи A і складе іспит:

$$P(E \cap A) = P(E \mid A)P(A) = 0.8 \cdot 0.9 = 0.72.$$

Отже, **72%** усіх студентів належать до групи A і складають іспит.

(б) За формулою повної ймовірності:

$$P(E) = P(E \mid A)P(A) + P(E \mid B)P(B) = 0.8 \cdot 0.9 + 0.3 \cdot 0.1 = 0.75.$$

Отже, випадково вибраний студент складає іспит з ймовірністю **75%**.

(в) За формулою Байєса:

$$P(B \mid E) = \frac{P(E \mid B)P(B)}{P(E)} = \frac{0.3 \cdot 0.1}{0.75} = 0.04.$$

Отже, серед студентів, які склали іспит, приблизно **4%** належать до групи B.

Частина 2. Барбекю

На барбекю-вечірці є п'ять вегетаріанців. Кожна порція овочів підгорає вщент з імовірністю 0.6. Зазначте припущення про незалежність підгоряння різних порцій та обчисліть імовірності того, що:

- **(а)** жодна з п'яти порцій не підгорить;
- **(б)** рівно дві з п'яти порцій підгорять;
- **(в)** принаймні три з п'яти порцій підгорять.

Розв'язання

Нехай X — кількість підгорілих порцій. Оскільки є 5 порцій, кожна підгорає незалежно з ймовірністю $p = 0.6$,

$$X \sim Bin(5, 0.6).$$

Для біноміального розподілу:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

(а) Жодна порція не підгорить

$$P(X = 0) = (1 - 0.6)^5 = 0.4^5 = 0.01024.$$

Отже, ймовірність того, що жодна порція не підгорить, становить приблизно **1.02%**.

(б) Рівно дві порції підгорять

$$P(X = 2) = \binom{5}{2} (0.6)^2 (0.4)^3.$$

$$P(X = 2) = 10 \cdot 0.36 \cdot 0.064 = 0.2304.$$

Отже, ймовірність того, що рівно дві порції підгорять, становить **23.04%**.

(в) Принаймні три порції підгорять

$$P(X \geq 3) = P(X = 3) + P(X = 4) + P(X = 5).$$

$$P(X \geq 3) = 0.68256.$$

Отже, ймовірність того, що підгорять принаймні три з п'яти порцій, становить приблизно **68.26%**.

Частина 3. Нормальний розподіл дохідності та портфель

Щотижнева дохідність акції X має нормальний розподіл із середнім $\mu = 0.03$ та **дисперсією** $\sigma^2 = 0.02$. Отже, стандартне відхилення дорівнює $\sqrt{0.02}$, а не 0.02.

- **(а)** Обчисліть $P(X > 0.01)$, $P(X \geq 0.01)$, $P(X < 0.01)$ та $P(0.01 < X < 0.03)$.
- **(б)** У 10 таких незалежних акцій інвестовано порівну 40% усього капіталу, а решту 60% — у безризиковий актив із дохідністю $r_f = 0.01$. Визначте вид розподілу дохідності портфеля та обчисліть його параметри. Вага кожної акції становить $0.4/10$.
- **(в)** Поясніть вплив кореляції між активами на дисперсію інвестицій. Наведіть формулу дисперсії портфеля з коваріаційними доданками.

```
# Параметри розподілу дохідності акції
stock_mean = 0.03
stock_variance = 0.02
stock_std = np.sqrt(stock_variance)

# Створюємо нормальний розподіл з заданими параметрами
stock_distribution = stats.norm(
    loc=stock_mean,
    scale=stock_std
)

# P(X > 0.01)
prob_gt_001 = 1 - stock_distribution.cdf(0.01)

# Для неперервного розподілу P(X >= 0.01) = P(X > 0.01),
# оскільки P(X = 0.01) = 0.
prob_ge_001 = prob_gt_001

# P(X < 0.01)
prob_lt_001 = stock_distribution.cdf(0.01)

# P(0.01 < X < 0.03)
prob_between = (
    stock_distribution.cdf(0.03)
    - stock_distribution.cdf(0.01)
)

print(f"P(X > 0.01) = {prob_gt_001:.4f}")
print(f"P(X >= 0.01) = {prob_ge_001:.4f}")
print(f"P(X < 0.01) = {prob_lt_001:.4f}")
print(f"P(0.01 < X < 0.03) = {prob_between:.4f}")
```

```
P(X > 0.01) = 0.5562
P(X >= 0.01) = 0.5562
P(X < 0.01) = 0.4438
P(0.01 < X < 0.03) = 0.0562
```

Частина 3(б). Дохідність портфеля

Позначимо дохідності акцій через $X_1, \ldots, X_{10}$. Вага кожної акції дорівнює $0.4/10 = 0.04$, тому дохідність портфеля:

$$R_p = 0.04 \sum_{i=1}^{10} X_i + 0.6 \cdot 0.01.$$

Сума незалежних нормально розподілених величин також має нормальний розподіл. Безризикова частина додає сталу дохідність, тому R_p має нормальний розподіл.

Його середнє:

$$E[R_p] = 0.04 \cdot 10 \cdot 0.03 + 0.6 \cdot 0.01 = 0.018.$$

Через незалежність акцій їхні коваріації дорівнюють нулю. Безризиковий актив має нульову дисперсію, тому

$$\mathrm{Var}(R_p) = 10 \cdot (0.04)^2 \cdot 0.02 = 0.00032.$$

Отже, $R_p \sim N(0.018, 0.00032)$, де другий параметр — дисперсія. Очікувана щотижнева дохідність становить **1.8%**, а стандартне відхилення — $\sqrt{0.00032} \approx 0.01789$, тобто **1.79 відсоткового пункту**.

Частина 3(в). Вплив кореляції

Якщо дохідності активів залежать одна від одної, дисперсія портфеля обчислюється так:

$$\mathrm{Var}(R_p) = \sum_i w_i^2 \sigma_i^2 + 2 \sum_{i<j} w_i w_j \,\mathrm{Cov}(X_i, X_j), \qquad \mathrm{Cov}(X_i, X_j) = \rho_{ij} \sigma_i \sigma_j.$$

Тут w_i — вага активу, σ_i — стандартне відхилення його дохідності, а ρ_{ij} — кореляція дохідностей двох активів. Безризиковий актив не додає дисперсії чи коваріації.

За додатних ваг додатна кореляція збільшує дисперсію портфеля: ціни активів частіше рухаються в одному напрямку. Від'ємна кореляція зменшує її, бо падіння дохідності одного активу може частково компенсуватися зростанням іншого. Тому розподіл коштів між активами зменшує ризик сильніше, коли їхні дохідності мають слабку або від'ємну кореляцію. У пункті (б) акції незалежні, тому всі коваріаційні доданки дорівнюють нулю.

Частина 4. Щотижнева дохідність Tesla

За допомогою Python (наприклад, `yfinance`) завантажте щотижневі ціни Tesla Inc. (`TSLA`) **за останні два роки** та обчисліть щотижневу дохідність. Зафіксуйте дату завантаження, точні межі дворічного періоду, вибір скоригованих або нескоригованих цін і спосіб обробки неповних тижнів. Збережіть копію даних у `data/` для відтворюваності.

- **(а)** За емпіричним розподілом оцініть імовірності дохідності **строго нижче** -2% та -4%.
- **(б)** Оцініть ті самі ймовірності за нормальним розподілом із параметрами, що дорівнюють вибірковому середньому та вибірковій дисперсії дохідності.
- **(в)** Порівняйте результати й обговоріть переваги та недоліки обох методів.

Для емпіричних оцінок врахуйте строгість нерівності на порозі; дохідність обчислюйте як відносну зміну ціни між послідовними тижнями.

```
import yfinance as yf

# Фіксуємо дату завантаження, щоб результати можна було відтворити.
download_date = pd.Timestamp("2026-10-07")

# Точний двохрічний період.
period_start = download_date - pd.DateOffset(years=2)

# yfinance трактує end як невключну межу,
# тому додаємо один день.
period_end_exclusive = download_date + pd.Timedelta(days=1)

# Файл для локальної копії даних.
tsla_cache_path = DATA_DIR / "tsla_daily_2024-10-07_2026-10-07.csv"

if tsla_cache_path.exists():
    # Якщо дані вже завантажені раніше — читаємо локальну копію.
    tsla_daily = pd.read_csv(
        tsla_cache_path,
        header=[0, 1],
        index_col=0,
        parse_dates=True
    )
```

```
else:
    # Завантажуємо денні ціни Tesla.
    tsla_daily = yf.download(
        "TSLA",
        start=period_start.strftime("%Y-%m-%d"),
        end=period_end_exclusive.strftime("%Y-%m-%d"),
        auto_adjust=False,
        progress=False
    )

    # Зберігаємо копію для відтворюваності.
    tsla_daily.to_csv(tsla_cache_path)

print(f"Дата завантаження: {download_date.date()}")
print(f"Період: {period_start.date()} — {download_date.date()}")
```

```
Дата завантаження: 2026-10-07
Період: 2024-10-07 — 2026-10-07
```

```
# Беремо скориговану ціну закриття.
# Adj Close враховує корпоративні події, наприклад спліти.
if isinstance(tsla_daily.columns, pd.MultiIndex):
    tsla_adjusted_close = tsla_daily["Adj Close"]["TSLA"]
else:
    tsla_adjusted_close = tsla_daily["Adj Close"]

tsla_adjusted_close = tsla_adjusted_close.dropna()
```

```
# Визначаємо останню повністю завершену п'ятницю.
# Поточний неповний тиждень не використовуємо.
days_back = (download_date.weekday() - 4) % 7

if days_back == 0:
    days_back = 7

last_completed_friday = download_date - pd.Timedelta(days=days_back)

# Для кожного тижня беремо останню доступну торгову ціну.
# Якщо п'ятниця була вихідним, буде взята остання ціна цього тижня.
tsla_weekly_price = (
    tsla_adjusted_close
    .resample("W-FRI")
    .last()
    .dropna()
)

# Видаляємо неповний поточний тиждень.
tsla_weekly_price = tsla_weekly_price[
    tsla_weekly_price.index <= last_completed_friday
]

# Щотижнева дохідність як відносна зміна ціни
tsla_weekly_return = tsla_weekly_price.pct_change().dropna()

print(f"Останній використаний завершений тиждень: {last_completed_friday.date()}")
print(f"Кількість тижневих дохідностей: {len(tsla_weekly_return)}")
```

```
Останній використаний завершений тиждень: 2026-10-02
Кількість тижневих дохідностей: 103
```

```
# Строго нижче −2% та −4%, як вимагає умова.
empirical_below_2 = np.mean(tsla_weekly_return < -0.02)
empirical_below_4 = np.mean(tsla_weekly_return < -0.04)

print(
    f"Емпірична P(R < −2%) = "
    f"{empirical_below_2:.4f} ({empirical_below_2 * 100:.2f}%)"
)

print(
```

```
    f"Емпірична P(R < −4%) = "
    f"{empirical_below_4:.4f} ({empirical_below_4 * 100:.2f}%)"
)
```

```
Емпірична P(R < −2%) = 0.3398 (33.98%)
Емпірична P(R < −4%) = 0.2330 (23.30%)
```

```
# Вибіркові параметри тижневої дохідності
weekly_mean = tsla_weekly_return.mean()
weekly_variance = tsla_weekly_return.var(ddof=1)
weekly_std = np.sqrt(weekly_variance)

# Ймовірності за підігнаним нормальним розподілом
normal_below_2 = stats.norm.cdf(
    -0.02,
    loc=weekly_mean,
    scale=weekly_std
)

normal_below_4 = stats.norm.cdf(
    -0.04,
    loc=weekly_mean,
    scale=weekly_std
)

print(f"Вибіркове середнє = {weekly_mean:.4f}")
print(f"Вибіркова дисперсія = {weekly_variance:.6f}")
print(f"Вибіркове стандартне відхилення = {weekly_std:.4f}")

print(
    f"Нормальна P(R < −2%) = "
    f"{normal_below_2:.4f} ({normal_below_2 * 100:.2f}%)"
)

print(
    f"Нормальна P(R < −4%) = "
    f"{normal_below_4:.4f} ({normal_below_4 * 100:.2f}%)"
)
```

```
Вибіркове середнє = 0.0077
Вибіркова дисперсія = 0.005313
Вибіркове стандартне відхилення = 0.0729
Нормальна P(R < −2%) = 0.3519 (35.19%)
Нормальна P(R < −4%) = 0.2564 (25.64%)
```

```
probability_comparison = pd.DataFrame({
    "Емпірична": [
        empirical_below_2,
        empirical_below_4
    ],
    "Нормальна": [
        normal_below_2,
        normal_below_4
    ]
}, index=[
    "R < −2%",
    "R < −4%"
])

display((probability_comparison * 100).round(2).rename(
    columns={
        "Емпірична": "Емпірична, %",
        "Нормальна": "Нормальна, %"
    }
))
```

	Емпірична, %	Нормальна, %
R < -2%	33.98	35.19
R < -4%	23.30	25.64

Інтерпретація

Було використано **103 щотижневі дохідності TSLA** за період від 2024-10-07 до 2026-10-07. Останнім використаним завершеним тижнем є 2026-10-02.

За емпіричним розподілом:

$$P(R < -2\%) = 0.3398,$$

тобто приблизно **33.98%** спостережуваних тижнів мали дохідність нижче -2%.

$$P(R < -4\%) = 0.2330,$$

тобто приблизно **23.30%** тижнів мали дохідність нижче -4%.

Вибіркове середнє щотижневої дохідності становить **0.0077**, тобто приблизно **0.77%**, а вибіркове стандартне відхилення — **0.0729**, або приблизно **7.29%**.

Якщо припустити нормальний розподіл із цими параметрами, отримуємо:

$$P(R < -2\%) = 35.19\%,$$

$$P(R < -4\%) = 25.64\%.$$

Нормальна модель у цьому випадку дає дещо більші оцінки ймовірності падіння, ніж емпіричний метод: на **1.21 відсоткового пункту** для порогу -2% і на **2.34 відсоткового пункту** для порогу -4%.

Емпіричний метод напряму використовує фактичну частоту подій у вибірці й не робить припущення про форму розподілу, але його точність залежить від кількості спостережень. Нормальний метод використовує всю вибірку для оцінки середнього та дисперсії і дозволяє оцінювати будь-які пороги, але результат залежить від того, наскільки добре нормальний розподіл описує реальні дохідності TSLA.

Отже, в наших даних обидва методи дають близькі, але не однакові оцінки, причому нормальна модель дещо вище оцінює ймовірність негативних тижневих дохідностей.

Частина 5. Біноміальна модель ціни

Початкова ціна активу $S(0) = 10$. У кожному періоді ціна зростає з імовірністю 0.55, а дохідність при русі вгору та вниз дорівнює 0.02 і -0.01 відповідно. Зазначте припущення про незалежність рухів ціни між періодами.

(а) Обчисліть імовірності того, що ціна зросте:

1. п’ять періодів поспіль;
2. три періоди поспіль;
3. рівно у трьох із п’яти періодів;
4. рівно у десяти з п’ятнадцяти періодів.

(б) Обчисліть очікувану сукупну дохідність інвестиції після двох періодів, враховуючи послідовне множення коефіцієнтів зростання ціни.

Розв'язання

Нехай у кожному періоді ціна зростає з ймовірністю

$$p = 0.55,$$

а падає з ймовірністю

$$q = 1 - p = 0.45.$$

Припускаємо, що рухи ціни в різних періодах незалежні.

(а) Ймовірності зростання ціни 1. Ціна зростає п'ять періодів поспіль

Для цього зростання має відбутися у кожному з п'яти незалежних періодів:

$$P = 0.55^5 \approx 0.0503.$$

Отже, ймовірність становить приблизно **5.03%**.

2. Ціна зростає три періоди поспіль

$$P = 0.55^3 \approx 0.1664.$$

Отже, ймовірність становить приблизно **16.64%**.

3. Ціна зростає рівно у трьох із п'яти періодів

Кількість зростань має біноміальний розподіл:

$$X \sim Bin(5, 0.55).$$

Тому

$$P(X = 3) = \binom{5}{3}(0.55)^3(0.45)^2 \approx 0.3369.$$

Отже, ймовірність становить приблизно **33.69%**.

4. Ціна зростає рівно у десяти з п'ятнадцяти періодів

$$X \sim Bin(15, 0.55),$$

тому

$$P(X = 10) = \binom{15}{10}(0.55)^{10}(0.45)^5 \approx 0.1404.$$

Отже, ймовірність становить приблизно **14.04%**.

(б) Очікувана сукупна дохідність після двох періодів При зростанні ціна множиться на

$$1 + 0.02 = 1.02,$$

а при падінні — на

$$1 - 0.01 = 0.99.$$

Очікуваний коефіцієнт зміни ціни за один період:

$$E[M] = 0.55 \cdot 1.02 + 0.45 \cdot 0.99 = 1.0065.$$

Через незалежність двох періодів очікуваний коефіцієнт зміни за два періоди:

$$E[M_1 M_2] = E[M_1]E[M_2] = 1.0065^2 = 1.01304225.$$

Тому очікувана сукупна дохідність:

$$1.01304225 - 1 = 0.01304225.$$

Отже, очікувана дохідність інвестиції після двох періодів становить приблизно **1.30%**.

Частина 6. Неперервні розподіли

Для кожного з розподілів $N(0,1)$, t_2 (розподіл Стьюдента з 2 ступенями свободи), $\mathrm{Exp}(1)$ (експоненціальний розподіл з інтенсивністю $\lambda = 1$) та χ^2_3 (розподіл хі-квадрат із 3 ступенями свободи) обчисліть:

- **(а)** $P(1 < X < 1.5)$;
- **(б)** $P(1 \le X < 1.5)$;
- **(в)** $P(X > 1)$;
- **(г)** $P(X \le 1.5)$;
- **(г)** $P(X = 1.5)$.

Поясніть, як неперервність розподілів впливає на ймовірності в окремих точках та на включення меж інтервалу.

```
# Розподіли, які задані в умові
continuous_distributions = {
    "N(0,1)": stats.norm(loc=0, scale=1),
    "t(2)": stats.t(df=2),

    # Для Exp(λ=1) параметр scale = 1/λ = 1
    "Exp(1)": stats.expon(scale=1),

    # Хі-квадрат із 3 ступенями свободи
    "χ²(3)": stats.chi2(df=3)
}

continuous_results = []

for name, distribution in continuous_distributions.items():
    # (а) P(1 < X < 1.5)
    prob_1_to_15 = (
        distribution.cdf(1.5)
        - distribution.cdf(1)
    )

    # (б) Для неперервного розподілу включення точки X=1
    # не змінює ймовірність
    prob_1_inclusive_to_15 = prob_1_to_15

    # (в) P(X > 1)
    # sf(x) = 1 - F(x), тобто ймовірність правого хвоста
    prob_above_1 = distribution.sf(1)

    # (г) P(X <= 1.5)
    prob_below_15 = distribution.cdf(1.5)
```

```
    # (ґ) Для неперервної випадкової величини
    # ймовірність одного конкретного значення дорівнює 0
    prob_equal_15 = 0.0

    continuous_results.append({
        "Розподіл": name,
        "P(1 < X < 1.5)": prob_1_to_15,
        "P(1 ≤ X < 1.5)": prob_1_inclusive_to_15,
        "P(X > 1)": prob_above_1,
        "P(X ≤ 1.5)": prob_below_15,
        "P(X = 1.5)": prob_equal_15
    })

continuous_results_df = pd.DataFrame(continuous_results)

display(
    continuous_results_df.set_index("Розподіл").round(4)
)
```

Розподіл	P(1 < X < 1.5)	P(1 ≤ X < 1.5)	P(X > 1)	P(X ≤ 1.5)	P(X = 1.5)
N(0,1)	0.0918	0.0918	0.1587	0.9332	0.0
t(2)	0.0751	0.0751	0.2113	0.8638	0.0
Exp(1)	0.1447	0.1447	0.3679	0.7769	0.0
$\chi^2(3)$	0.1190	0.1190	0.8013	0.3177	0.0

Інтерпретація

Для всіх чотирьох розподілів виконується

$$P(1 < X < 1.5) = P(1 \leq X < 1.5),$$

оскільки всі вони є неперервними, а для неперервної випадкової величини ймовірність отримати одне конкретне значення дорівнює нулю:

$$P(X = 1) = P(X = 1.5) = 0.$$

Тому включення або невключення межі інтервалу не змінює його ймовірність.

Для стандартного нормального розподілу $N(0,1)$ маємо $P(X > 1) = 0.1587$, а для розподілу Стьюдента t_2 — 0.2113. Більша ймовірність у t_2 пов'язана з тим, що цей розподіл має важчі хвости.

Для $Exp(1)$ маємо $P(X > 1) = 0.3679$, а для χ^2_3 — 0.8013. Ці розподіли мають лише невід'ємні значення і є правосторонньо асиметричними, тому їхні ймовірності в заданих інтервалах суттєво відрізняються від симетричних $N(0,1)$ та t_2.

Головний висновок: у неперервних розподілах ймовірність визначається площею на інтервалі, а ймовірність окремої точки завжди дорівнює нулю.